Istio Hybrid: Multi-Cluster and VMs in a Service Mesh

Workshop on Istio in sidecar mode with the classic Istio APIs (VirtualService, DestinationRule, ServiceEntry, Gateway): several Kubernetes clusters and virtual machines in one shared mesh, on an IPv6-only network.

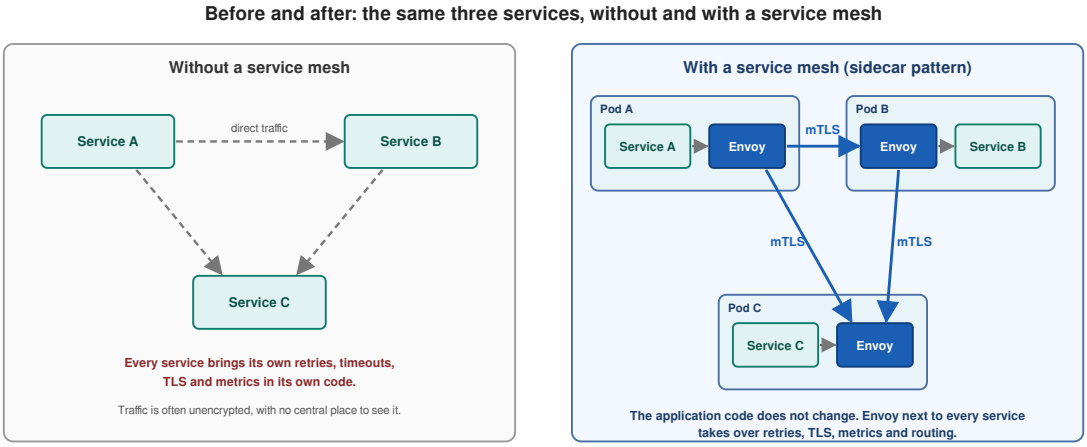
All material in this repository is in English.

Agenda

1. Fundamentals: service mesh and Istio
   - What is a service mesh, and what is Istio? Sidecar and ambient
   - Why a service mesh?
   - Challenges and benefits
   - Istio architecture and components
   - Istio ingress gateway vs. Kubernetes Gateway API
   - Ports of istiod and of the sidecar (sidecar mode)
   - How the sidecar gets into the pod (sidecar injection)
2. Explore the lab
   - Look around the lab: nodes, pods, sidecar, `istioctl proxy-status`
3. Ingress gateway on IPv6
   - Expose Bookinfo through the ingress gateway over IPv6
4. Two clusters, one mesh
   - Check the link between two clusters: `istioctl remote-clusters`
   - Answer requests with pods from the other cluster
5. Virtual machines in the mesh
   - WorkloadGroup, WorkloadEntry and EndpointSlice: who is who
   - Add a virtual machine to the mesh (IPv6 only, VM without IPv4)
   - Ports of istiod and of the sidecar on the VM
   - (Optional) Add a virtual machine to the mesh with IPv4 and IPv6 (dual stack)
   - (Optional) Dual stack on the VM: what to set (`DUAL_STACK` and `ISTIO_DUAL_STACK`)
   - (Optional) Reference: Dual stack on the VM: details and measurements
6. Debugging Istio
   - Logs and log levels
   - ControlZ: the debug page of istiod

Fundamentals: service mesh and Istio

What is a service mesh, and what is Istio? Sidecar and ambient



What is a service mesh?

A service mesh is a layer of infrastructure that handles the communication between services:

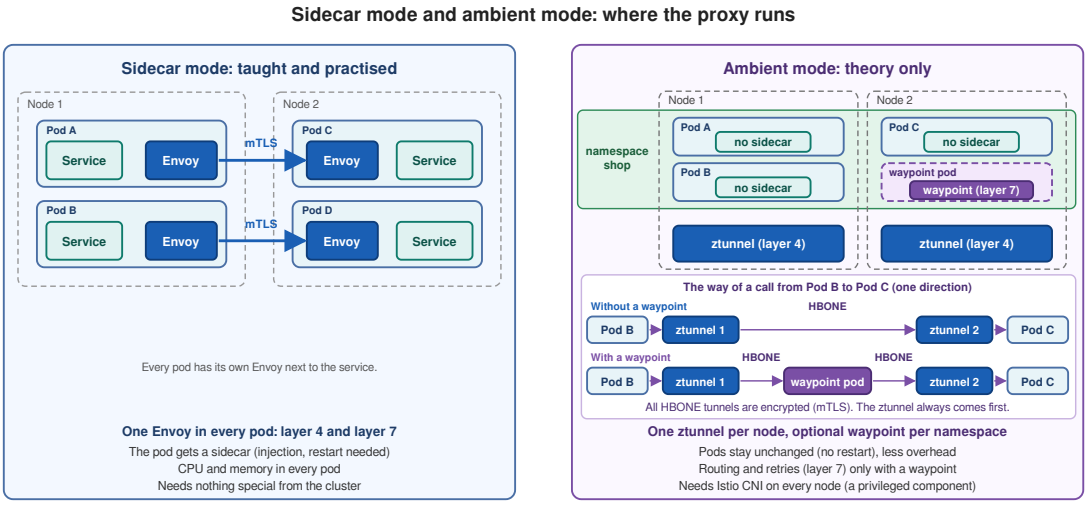
- It sits **between** the services, as a transparent layer. The application code does not change.
- Retries, timeouts, encryption (mTLS), access rules and metrics are configured **in one place**, not in every service.
- It gives applications zero-trust security, observability and advanced traffic management without code changes.

Istio in one paragraph

Istio is an open source service mesh. It was started by Google, IBM and Lyft. Its data path is made of **Envoy** proxies. In the classic model an Envoy runs next to every workload as a **sidecar** and catches all network traffic of that workload. A central control plane, **istiod**, tells every proxy what to do.

In this course you use Istio in **sidecar mode** with the classic Istio APIs (`VirtualService` , `DestinationRule` , `ServiceEntry` , `Gateway`).

Two architectures: sidecar and ambient



Istio knows two data plane modes. They can run side by side in one cluster.

	Sidecar mode	Ambient mode
Where is the proxy?	One Envoy **in every pod**	One `ztunnel` **per node** (layer 4), optional `waypoint` proxy (layer 7, usually one per namespace)
What the proxy does	Layer 4 and layer 7: mTLS, routing, retries,	`ztunnel`: mTLS and layer 4 telemetry. `waypoint`: routing, retries, layer 7 rules

	metrics	
Cost	CPU and memory in every pod	Less overhead, because proxies are shared
Pod changes	The pod gets a sidecar (injection, restart needed)	No change to the pod
What it needs from the cluster	Nothing special	An Istio node agent on every node (Istio CNI, a privileged component)
In this course	**Taught and practised**	**Theory only, about 15 minutes, no lab**

Why sidecar here: you work as a tenant and you cannot install a privileged component on every node. The sidecar needs nothing of that, and it is the model that is still in wide use. Ambient is worth knowing because it changes the data path, so you can judge it when the question comes up.

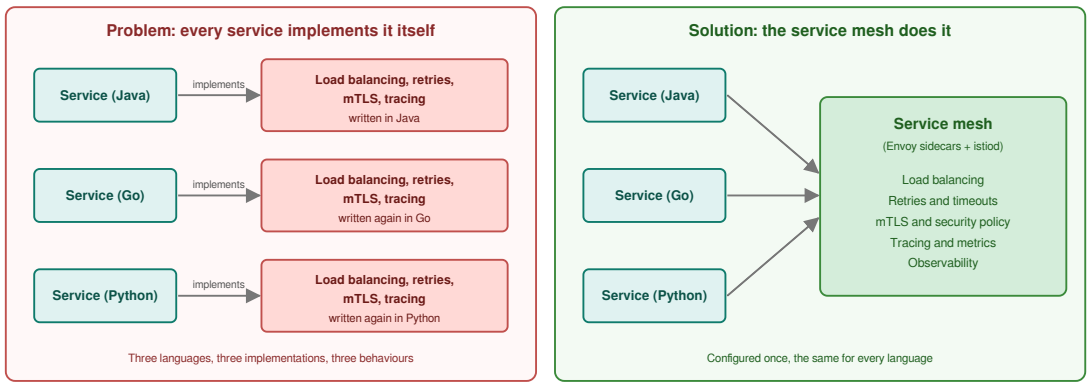
What you should remember

- A service mesh consists of a **control plane** (decides) and a **data plane** (the proxies that carry the traffic).
- In sidecar mode every pod has an Envoy next to the application.
- The mesh works at the network level, so it works for every language.

Next: Why a service mesh?

Why a service mesh?

Where the network features live: in every service, or once in the mesh



The problem

With a few services, the network is easy. With dozens or hundreds of services, communication becomes the hard part. Every service needs:

- load balancing
- retries and timeouts
- circuit breaking
- mTLS encryption
- distributed tracing and metrics

If every service implements this itself, you get:

- **Different behaviour per language.** The Java service retries three times, the Go service never retries, the Python service retries forever.
- **High maintenance.** A change to the TLS settings means a new release of every service.
- **No overview.** Nobody can say which service talks to which, and whether the traffic is encrypted.

The solution

The mesh moves this work **out of the application code into the infrastructure**:

Who	What changes
Developers	Concentrate on the business logic
Platform team	Sets policy once (encryption, access, retries) for all services
Operations	Sees the same metrics for every service, whatever the language

The five things a mesh gives you

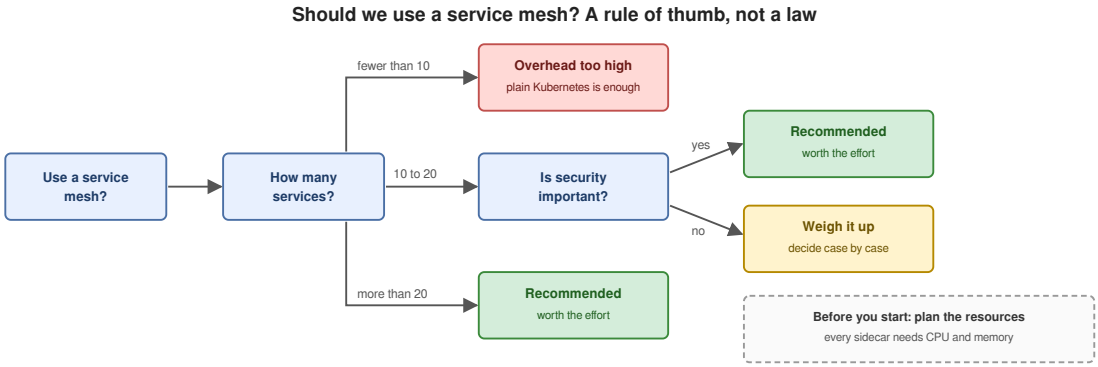
Area	What it means

Observability	Metrics, traces and logs for all service-to-service traffic
Traffic management	Canary releases, weighted routing, load balancing
Resilience	Retries, timeouts, outlier detection
Security	mTLS between services, authentication and authorization policy
Service discovery	Services find each other, also when the system changes

You will meet every row of this table in a later module.

Previous: What is a service mesh? Next: Challenges and benefits

Challenges and benefits



Benefits

- **mTLS between all services**, automatic. The sidecars get certificates and rotate them.
- **Traffic control**: canary release, blue-green, A/B tests, weighted routing.
- **One view of everything**: metrics, traces and logs look the same for every service.
- **Central security policy**: who may call whom is a configuration, not code.
- **No code changes.**

Challenges

Challenge	What it means in practice
Resource overhead	Every sidecar needs CPU and memory. With many pods this adds up. You set requests and limits for the sidecar.
Extra latency	Every call goes through two proxies, the one of the caller and the one of the target. The delay is small, but it is there.
Learning curve	New objects, new terms, new places where a problem can hide.
Harder debugging	A failed call can be the application, the sidecar, the network or the mesh configuration. Tools like `istioctl proxy-status` help.
Another system to run	istiod, certificates, upgrades.

When is it worth it?

A rule of thumb, not a law:

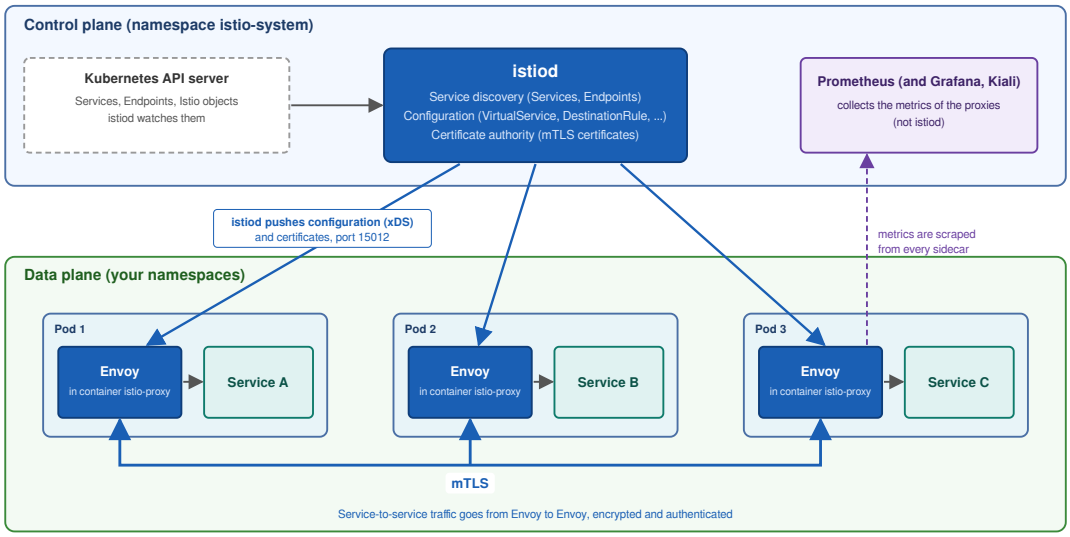
- from about 20 to 30 services
- high requirements for security and compliance (for example encryption between all services, proof of who may call whom)
- many teams that use different languages

With fewer than 10 services the overhead usually outweighs the gain. Between 10 and 20 it depends mainly on how much security matters to you.

Previous: Why a service mesh? Next: Istio architecture and components

Istio architecture and components

Istio in sidecar mode: control plane and data plane



An Istio mesh has two parts:

Part	What it is	Where in your lab
Control plane	`istiod`: one Deployment that decides	namespace `istio-system`
Data plane	The Envoy proxies that carry the traffic: one in every pod of the mesh, plus the gateways	the sidecar in every pod, the gateway in `istio-ingress`

The control plane: istiod

`istiod` is one process with several jobs:

- **Service discovery.** It watches the Kubernetes API for Services and Endpoints.
- **Configuration.** It reads the Istio objects (`VirtualService` , `DestinationRule` , `Gateway` , ...) and translates them into Envoy configuration.
- **Certificate authority.** It signs the workload certificates for mTLS. The identity of a workload is its Kubernetes ServiceAccount.
- **Distribution.** It pushes the configuration to all proxies over a gRPC connection (xDS, port 15012).

istiod does **not** carry any application traffic, and it does not collect metrics. The proxies expose their metrics, and Prometheus scrapes them.

The data plane: Envoy

Envoy is a fast, generic proxy. In a pod it runs as the container `istio-proxy` . Inside this container two programs run:

Program	Job
`pilot-agent` (process 1)	Connects to istiod, fetches the certificate, starts Envoy
`envoy` (child process)	Carries the traffic: routing, retries, mTLS, metrics

Envoy does not talk to istiod directly. It talks to `pilot-agent` over a local Unix socket, and `pilot-agent` holds the connection to istiod. A small init container, `istio-init` , runs once at pod start and sets up iptables rules. These rules send all traffic of the pod through Envoy.

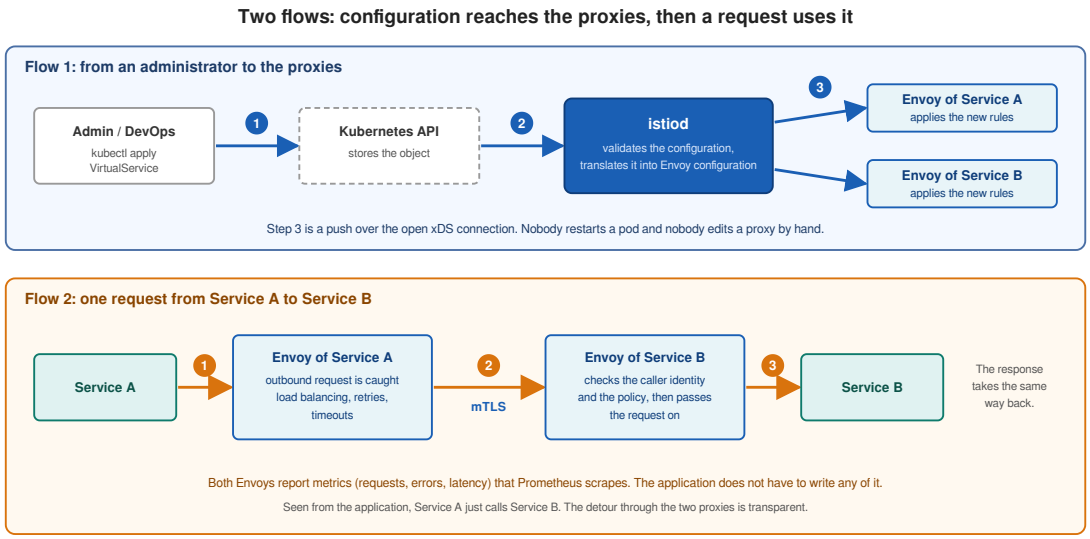
The messages that carry configuration are called **xDS**. You will see the names in `istioctl proxy-status` :

Name	What istiod sends
CDS	clusters: the services a proxy can call
EDS	endpoints: the pod addresses behind a service
LDS	listeners: which ports the proxy listens on
RDS	routes: how requests are matched and where they go
NDS	names: the DNS table for the DNS proxy

Gateways

A gateway is an Envoy that runs **on its own**, without an application next to it. It stands at the edge of the mesh. The **ingress gateway** lets traffic in, an egress gateway lets traffic out in a controlled way. How an ingress gateway is configured is the topic of the next page.

From configuration to traffic



1. You apply a high level object, for example a `VirtualService`.
2. istiod validates it and translates it into Envoy configuration.
3. istiod pushes the result to every proxy that needs it. Nobody restarts a pod.
4. At run time the Envoy of the caller catches the outbound request, applies load balancing and retries, and sends it with mTLS to the Envoy of the target. That Envoy checks the identity and the policy and hands the request to the application.

What you should remember

- istiod **decides**, Envoy **carries**.
- The configuration travels from istiod to Envoy, never the other way round. The connection always starts at the proxy.
- The proxies hold their last configuration. If istiod is down for a while, existing traffic keeps flowing, but changes and new certificates stop.

Previous: Challenges and benefits Next: Istio ingress gateway vs. Kubernetes Gateway API

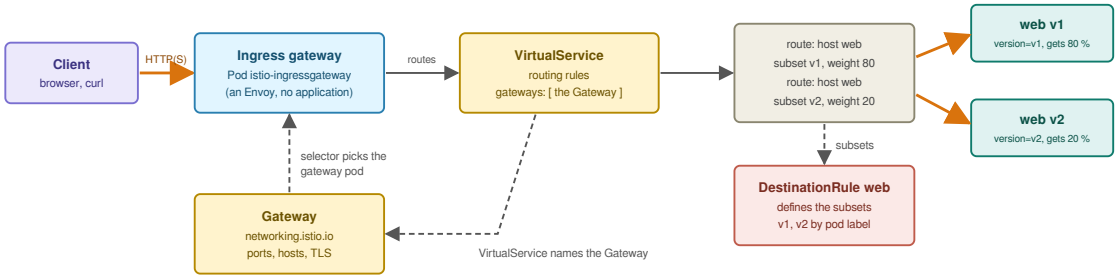
Istio ingress gateway vs. Kubernetes Gateway API

Traffic that comes **into** the cluster from outside is called north-south traffic. In Istio you have two ways to describe it:

- the **Istio APIs**: `Gateway` plus `VirtualService` (plus `DestinationRule`). This is what the exercise Expose Bookinfo through the ingress gateway over IPv6 uses.
- the **Kubernetes Gateway API**: `GatewayClass`, `Gateway` and `HTTPRoute`. It is a Kubernetes standard, not an Istio invention, and several products implement it.

The Istio way

Istio ingress gateway: Gateway, VirtualService and DestinationRule



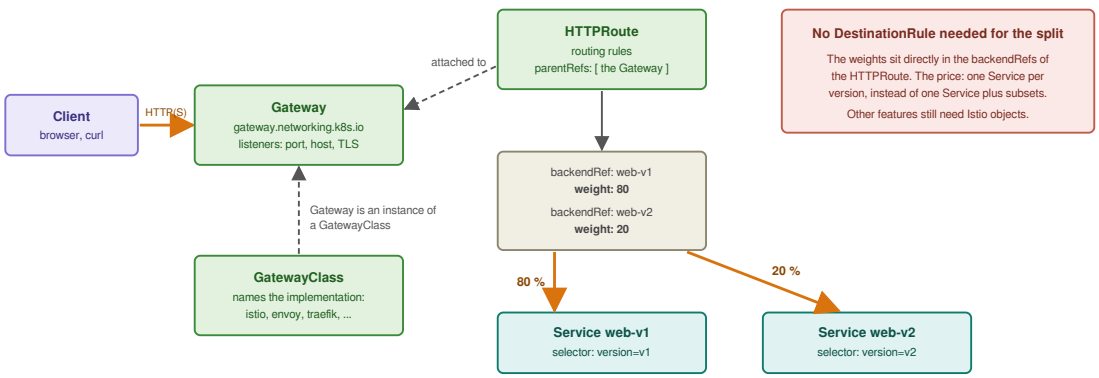
The Gateway only opens the door (port, host, TLS). Where the traffic goes is decided in the VirtualService, the subsets come from the DestinationRule.
Three Istio objects, plus the gateway pod that has to exist already.

Object	What it does
Ingress gateway (`istio-ingressgateway`)	A Deployment with a Service of type `LoadBalancer`. In the lab both are called `istio-ingressgateway` and live in the namespace `istio-ingress`.
`Gateway` (`networking.istio.io`)	Opens the door on the ingress gateway: ports, hosts, TLS. A `selector` picks the gateway pods
`VirtualService`	The routing rules. It names the `Gateway` in its `gateways` field
`DestinationRule`	Defines **subsets** (for example `v1`, `v2` by pod label) and the policy for the target: load balancing, connection pool, outlier detection

The same `VirtualService` and `DestinationRule` also work for traffic **inside** the mesh. That is why you learn them first.

The Kubernetes Gateway API way

Kubernetes Gateway API: GatewayClass, Gateway and HTTPRoute



The Gateway opens the door, the HTTPRoute decides where traffic goes. Different people can own the two objects.

Object	What it does
`GatewayClass`	Names the implementation (`istio`, `envoy`, `traefik`, ...). One per implementation, usually created by the platform
`Gateway` (`gateway.networking.k8s.io`)	An instance of a class: the listeners (port, host, TLS)
`HTTPRoute`	The routing rules. It attaches to a `Gateway` with `parentRefs`

Two differences to remember:

- **The weights are in the route.** An `HTTPRoute` lists `backendRefs` with a `weight`. Each backend is a Service, so a canary needs one Service per version. No `DestinationRule` is needed for the split.

- **Istio can create the gateway pod for you.** With the Gateway API, a `Gateway` of class `istio` makes Istio create the Deployment and the Service itself. With the Istio `Gateway` you run the gateway pod yourself.

Other route types exist, for example `GRPCRoute` for gRPC.

Side by side

	Istio `Gateway` + `VirtualService`	**Gateway API `Gateway` + `HTTPRoute`**
Who defines it	Istio	Kubernetes SIG Network, many implementations
Door and routing	`Gateway` and `VirtualService`	`Gateway` and `HTTPRoute`
Link between them	`VirtualService.gateways` names the `Gateway`	`HTTPRoute.parentRefs` names the `Gateway`
Canary	weights over **subsets** (needs a `DestinationRule`)	weights over **Services** in `backendRefs`
Gateway pod	you run it	Istio can create it from the `Gateway`
Who may attach a route	a `VirtualService` that names the `Gateway`	the `Gateway` decides with `allowedRoutes` (namespaces, kinds)
Works with another product	no, Istio only	yes, same objects with another `GatewayClass`
Where it is installed	Istio brings the CRDs	The Gateway API CRDs are **not** part of Kubernetes, someone has to install them

The last row matters if you are a tenant: installing CRDs is a cluster-wide act. Ask your trainer whether the Gateway API CRDs are installed in your lab.

Which one to use?

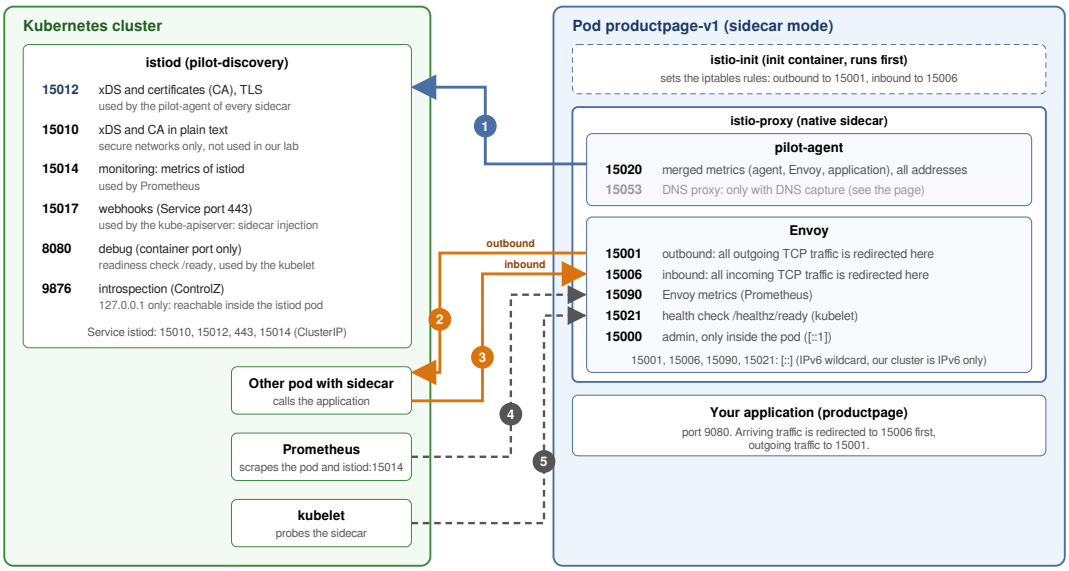
- The Gateway API is the direction Istio is moving in. For **new ingress** configuration it is a good default when the CRDs are available.
- The Istio APIs stay important. Not every feature of `VirtualService` and `DestinationRule` has an equivalent in the Gateway API, for example fault injection, connection pool settings and outlier detection are still configured with the Istio objects. Existing setups are full of them, and many companies run the classic APIs.
- You can use both in one cluster, and you can move step by step.
- **One owner per Service.** Do not describe the routing of the same Service with an `HTTPRoute` and a `VirtualService` at the same time. Pick one. (Hint: It does not work properly)

In this course the focus is on the Istio APIs. You see the Gateway API as a short contrast.

Previous: Istio architecture and components Next: Ports of istiod and of the sidecar

Ports of istiod and of the sidecar (sidecar mode)

Ports of istiod and of the sidecar in a pod: who listens where, who connects to whom



1 The sidecar starts. The pilot-agent connects to istiod on 15012: it gets the configuration (xDS) and the workload certificate.
The sidecar opens the connection, istiod never connects to the pod.

2 Outbound: the application calls another pod. iptables sends the connection to Envoy on 15001, Envoy sends it with mTLS to the other pod.

3 Inbound: another pod calls the application. The mTLS connection arrives, iptables redirects it to Envoy on 15006, Envoy ends the mTLS and hands the request to the application (here 9080).

4 Prometheus scrapes 15020 (everything merged) or 15090 (Envoy only) of the pod, and 15014 of istiod. These ports are not redirected.

5 The kubelet asks 15021 (/healthz/ready) as startup and readiness probe of istio-proxy. This port is not redirected either.

Not drawn: the kube-apiserver, which calls istiod on 15017 when a pod is created (sidecar injection, see the injection page), the other cluster and the gateways.
Documented by Istio but not listening in our pod: 15002, 15004, 15008. Measured in our lab: Istio 1.29.9, IPv6-only cluster, pod productpage-v1.

The picture shows every port, its task and who connects to it. Measured in our lab: Istio 1.29.9, IPv6 only, the `productpage` pod of Bookinfo. The tasks come from the Istio documentation (*Ports used by Istio*). `istiod` is the program `pilot-discovery`. The container `istio-proxy` runs `pilot-agent` and Envoy; in our lab it is a Kubernetes *native sidecar* (listed under `initContainers` with `restartPolicy: Always`).

Not in the picture

- **Addresses:** `[::]` is the IPv6 wildcard address (our cluster is IPv6 only). `15020` listens on all addresses, `15000` on `[::1]` only.
- **Prometheus:** the pod carries the annotations `prometheus.io/scrape: "true"`, `prometheus.io/port: "15020"` and `prometheus.io/path: /stats/prometheus`. `15020` gives everything in one place, `15090` only the metrics of Envoy.
- **Not listening** in our pod: `15002` (failure detection), `15004` (debug), `15008` (HBONE tunnel, ambient mode).
- **The `Service istiod`** has `15010`, `15012`, `443` (forwarded to `15017`) and `15014`. `8080` is a container port only.

What iptables redirects

The init container `istio-init` sets these rules in the pod (we read them from its log):

Direction	Rule
outbound	all TCP to `15001`
inbound	all TCP to `15006`
inbound, **not** redirected	`15008`, `15090`, `15021`, `15020`: the ports of the sidecar itself
outbound, **not** redirected	traffic of the proxy itself (user and group `1337`) and traffic to `127.0.0.1`, otherwise Envoy would redirect its own traffic to itself

DNS capture and port 15053

Normally an application asks the cluster DNS (`kube-dns`). It knows the Kubernetes `Service`s, but not your own `ServiceEntry` hosts or services outside of Kubernetes. With **DNS capture** the sidecar answers itself: "all DNS requests from an application will be redirected to the sidecar". The `pilot-agent` keeps a table of names and addresses from `istiod` and answers what it can. Any other name goes on to the DNS server from `/etc/resolv.conf`. It also gives addresses to `ServiceEntry`s that have none (*auto allocation*, `240.240.0.0/16`), and a VM uses it to find the services of the cluster.

- **Off by default in sidecar mode** (on by default in ambient mode from Istio 1.25 on).
- **Mesh-wide:** `ISTIO_META_DNS_CAPTURE: "true"` in `meshConfig.defaultConfig.proxyMetadata` (ConfigMap `istio`). **One pod:** the annotation `proxy.istio.io/config`.
- **In our lab** the mesh configuration of cluster a has `ISTIO_META_DNS_CAPTURE` and `ISTIO_META_DNS_AUTO_ALLOCATE` set to `"true"`, because the VM exercise switches them on.

- **It is read when the pod is created.** We measured it:

Pod	Started	Listens on `15053`
`productpage-v1`	before DNS capture was switched on	no
a new test pod in a labelled namespace	after	yes: `[::1]:15053`, tcp and udp

That is why the picture, measured on `productpage-v1`, shows `15053` greyed out.

What the words mean

Word	Meaning
xDS	The family of discovery APIs over which `istiod` sends the configuration to the proxies: clusters, listeners, endpoints, routes and names (CDS, LDS, EDS, RDS, NDS). You see them in the column `SUBSCRIBED TYPES` of `istioctl proxy-status`. One long-lived gRPC connection that the proxy opens. **ADS** is the one stream over which `istiod` sends all of this to the proxy; the log scope `ads` is named after it (see log levels)
CA	`istiod` is also the certificate authority. The `pilot-agent` sends a certificate request with the token of the service account of the pod (the volume `istio-token`) on the same connection and gets the workload certificate for mTLS
`15012` and `15010`	The same two services, with TLS (`15012`) and in plain text (`15010`). Plain text is only for networks you trust completely. We use `15012`
Webhook	An HTTPS address that the `kube-apiserver` calls when something is created. A *mutating* webhook may change the object (the sidecar injection), a *validating* webhook may reject it (`istiod` checks Istio resources). Both use `443` and `15017`
Readiness (`8080`)	The kubelet asks `/ready` to find out whether `istiod` is ready. Only then the `Service` sends traffic to it
ControlZ (`9876`)	The debug page of `istiod`: it shows its environment, start arguments, log scopes and memory. It listens on `127.0.0.1` only. `istioctl admin log` talks to `istiod` through it. See ControlZ

Look at it yourself

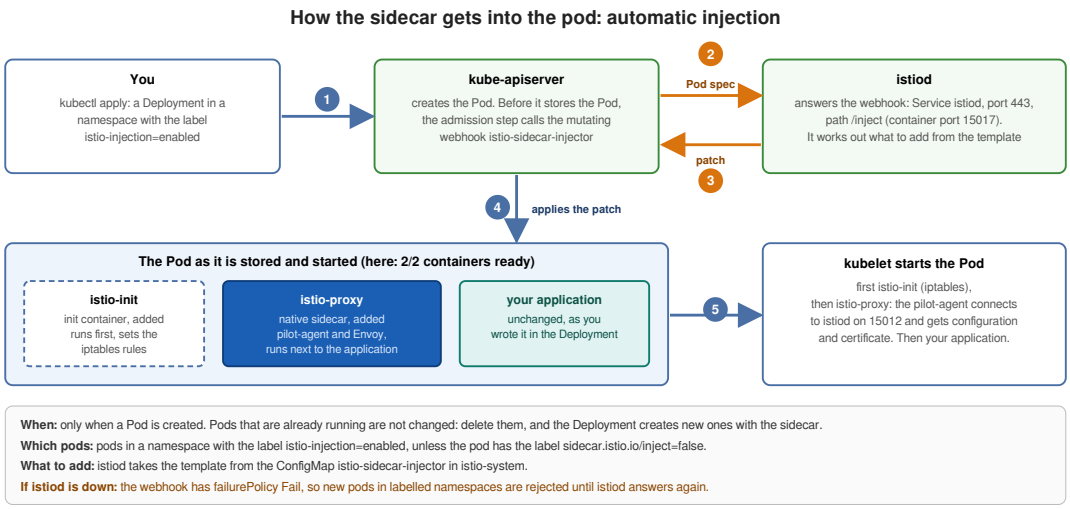
The commands run on your bastion host as your lab user. They use the `productpage` pod of Bookinfo.

```
kubectl exec deploy/productpage-v1 -c istio-proxy -- ss -ltnp
kubectl logs deploy/productpage-v1 -c istio-init | grep -E "^-A ISTIO_(INBOUND|REDIRECT|IN_REDIRECT)"
kubectl get pod -l app=productpage -o jsonpath='{.items[0].metadata.annotations}'
kubectl -n istio-system get deploy istiod -o jsonpath='{range .spec.template.spec.containers[0].ports[*]}{.name}=
{.containerPort} {end}'
kubectl -n istio-system get svc istiod
kubectl -n istio-system get cm istio -o jsonpath='{.data.mesh}' | grep ISTIO_META_DNS
```

The same ports on a virtual machine: Ports of istiod and of the sidecar on the VM.

Previous: Istio ingress gateway vs. Kubernetes Gateway API Next: How the sidecar gets into the pod (sidecar injection)

How the sidecar gets into the pod (sidecar injection)



The picture shows the steps. This page adds what we measured in our lab (Istio 1.29.9).

Which pods get a sidecar

What you do	Result	Seen in our lab
namespace without a label	no sidecar	pod `a`: `1/1`, no init containers
namespace with `istio-injection=enabled`	sidecar	pod `b`: `2/2`, init containers `istio-init` and `istio-proxy`
namespace with the label, pod with `sidecar.istio.io/inject=false`	no sidecar	pod `c`: no init containers

With control plane revisions the label `istio.io/rev` is used instead (if both are set, `istio-injection` takes precedence). If either label says "disabled", the pod is not injected.

Running pods are not changed: pod `a` was created before the namespace got its label and stayed `1/1`. Delete the pods, and the Deployment creates new ones with the sidecar.

What the injector adds

The annotation `sidecar.istio.io/status` of a pod lists it:

- the init container `istio-init` and the container `istio-proxy` (also listed under `initContainers`, with `restartPolicy: Always`: a Kubernetes *native sidecar*)
- volumes, among them `istio-envoy`, `istio-token`, `istiod-ca-cert` and `workload-certs`

`kubectl get pod` shows `2/2`: your application and the sidecar. Your Deployment stays unchanged.

The `MutatingWebhookConfiguration` `istio-sidecar-injector` has four webhooks, one for each way to select pods (label `istio-injection` or `istio.io/rev`, on the namespace or on the pod). All four call the same `Service`.

Look at it yourself

The commands run on your bastion host as your lab user.

```
kubectl get ns default --show-labels
kubectl get mutatingwebhookconfiguration istio-sidecar-injector -o jsonpath='{range .webhooks[*]}{.name}{" -> "}
{.clientConfig.service.name}{":"}{.clientConfig.service.port}{.clientConfig.service.path}{" failurePolicy="}
{.failurePolicy}{"\n"}{end}'
kubectl get pod -l app=productpage
kubectl get pod -l app=productpage -o jsonpath='{.items[0].metadata.annotations.sidecar\.istio\.io/status}'
```

Try it: three pods, three results

```
kubectl create namespace inj-test
kubectl -n inj-test run a --image=curlimages/curl:8.11.1 --restart=Never -- sleep 3600
kubectl label namespace inj-test istio-injection=enabled
kubectl -n inj-test run b --image=curlimages/curl:8.11.1 --restart=Never -- sleep 3600
kubectl -n inj-test run c --image=curlimages/curl:8.11.1 --restart=Never --labels=sidecar.istio.io/inject=false --
sleep 3600
kubectl -n inj-test wait --for=condition=Ready pod --all --timeout=120s
kubectl -n inj-test get pod
kubectl -n inj-test get pod a b c -o custom-columns=NAME:.metadata.name,INIT:.spec.initContainers[*].name
```

```
NAME   READY   STATUS    RESTARTS   AGE
a      1/1     Running   0          6s
b      2/2     Running   0          5s
c      1/1     Running   0          5s

NAME   INIT
a      <none>
b      istio-init,istio-proxy
c      <none>
```

Clean up:

```
kubectl delete namespace inj-test
```

Previous: Ports of istiod and of the sidecar

Explore the lab

Look around the lab: nodes, pods, sidecar, `istioctl proxy-status`

Background

Your lab already has Istio installed in sidecar mode. Nothing is left for you to install. In this exercise you look at what is running, and you learn the one command you will use all week: `istioctl proxy-status`.

The lab consists of:

Part	What it is
Kubernetes cluster	one control-plane node and one worker node, IPv6 only
Istio	control plane `istiod` in the namespace `istio-system`, sidecar mode
Namespace `app`	two pods, `web` and `client`, both with a sidecar
Namespace `vm`	a virtual machine (`diameter-adapter`) that is part of the mesh

All commands below run on your bastion host.

Step 1: Look at the nodes

```
kubectl get nodes -o wide
```

Expected output (addresses differ in your lab):

```
NAME    STATUS   ROLES           AGE   VERSION    INTERNAL-IP
h6-cp   Ready    control-plane   ...   v1.35.10   2a01:4f8:c016:aa79::1
h6-w    Ready    <none>          ...   v1.35.10   2a01:4f8:c016:17b5::1
```

Note that the node addresses are IPv6 addresses. There is no IPv4 address inside the cluster.

Step 2: Look at the pods in the mesh

```
kubectl get pods -n app -o wide
```

```
NAME     READY   STATUS    RESTARTS   AGE   IP
client   2/2     Running   0          ...   2a01:4f8:c016:aa79::1:e5c6
web      2/2     Running   0          ...   2a01:4f8:c016:aa79::1:e5c5
```

Both pods show `2/2`, although each pod has only one application container. The second container is the Istio sidecar (Envoy). Each pod has its own IPv6 address, there is no NAT between pods.

Step 3: Find the sidecar

```
kubectl get pod web -n app -o jsonpath='{.spec.containers[*].name}'
```

```
web
```

The sidecar is not in this list. In this lab (Kubernetes 1.35) Istio runs the sidecar as a native sidecar: a container in `initContainers` that has `restartPolicy: Always`.

```
kubectl get pod web -n app -o jsonpath='{.spec.initContainers[*].name}'
```

```
istio-init istio-proxy
```

`istio-init` sets up the traffic redirection and ends. `istio-proxy` is the sidecar and keeps running.

Step 4: Ask Istio who is connected

```
istioctl proxy-status
```

```
NAME           CLUSTER        ISTIOD                     VERSION     SUBSCRIBED TYPES
client.app     cluster-h6     istiod-8ff6d656c-v555p     1.29.9      5 (CDS,LDS,EDS,RDS,NDS)
h6-vm.vm       cluster-h6     istiod-8ff6d656c-v555p     1.29.9      5 (CDS,LDS,EDS,RDS,NDS)
web.app        cluster-h6     istiod-8ff6d656c-v555p     1.29.9      5 (CDS,LDS,EDS,RDS,NDS)
```

This is the list of all proxies that are currently connected to `istiod`. Read it column by column:

Column	Meaning
NAME	the proxy: pod name (or VM name) and namespace

CLUSTER	the cluster it belongs to
ISTIOD	the `istiod` pod that serves it
VERSION	the proxy version, after an upgrade you see here who is still old
SUBSCRIBED TYPES	which configuration `istiod` pushes to it (see below)

The abbreviations are the kinds of configuration Envoy receives from `istiod`:

Abbreviation	Meaning
CDS	clusters: where can this proxy send traffic
LDS	listeners: on which ports does it listen
EDS	endpoints: which addresses are behind a service
RDS	routes: how are requests distributed
NDS	name table: used by the DNS proxy, for example on the VM

Look for the VM: `h6-vm.vm` is in the list, so the virtual machine is part of the mesh like any pod.

Step 5: See the VM as a mesh member

```
kubectl get workloadentry -n vm
```

```
NAME                                        AGE   ADDRESS
diameter-adapter-2a01-4f8-c016-dab1--1-net1   ...   2a01:4f8:c016:dab1::1
```

A `WorkloadEntry` is how Istio describes a workload that is not a pod. This one was created automatically when the VM connected.

Step 6: Call the VM from a pod

```
kubectl exec -n app client -c client -- curl -s http://diameter-adapter.vm.svc.cluster.local:8000/ | head -1
```

```
VM diameter-adapter (IPv6), client=::6
```

The request goes from the sidecar of `client` to the sidecar of the VM, encrypted with mutual TLS. The program on the VM answers.

Step 7: Watch a proxy appear and disappear

Start a new pod in a namespace with sidecar injection:

```
kubectl run extra -n app --image=nginx:1.27
kubectl wait -n app --for=condition=Ready pod/extra --timeout=120s
istioctl proxy-status
```

`extra.app` is now in the list.

Now start a pod in a namespace without sidecar injection and compare:

```
kubectl create namespace plain
kubectl run plain-web -n plain --image=nginx:1.27
kubectl wait -n plain --for=condition=Ready pod/plain-web --timeout=120s
kubectl get pod plain-web -n plain
istioctl proxy-status
```

The pod shows `1/1` and does not appear in `proxy-status`. It is not part of the mesh.

Take-away: `proxy-status` shows who is connected to `istiod`. It does not show whether everything works. A VM can be in the list and still not receive traffic, for example if its WorkloadEntry is missing. Later in the training you will use this list to find a VM that dropped out of the mesh.

Clean up

```
kubectl delete pod extra -n app
kubectl delete namespace plain
```

Summary

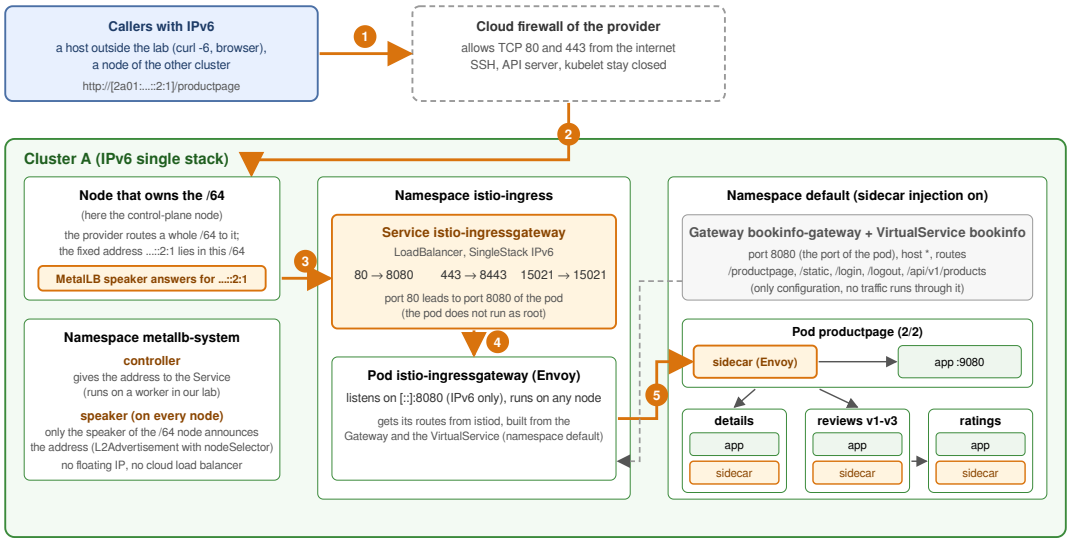
Question	Command
Which proxies are connected to istiod?	`istioctl proxy-status`

Is this pod in the mesh?	`READY 2/2` and the pod is in `proxy-status`
Where is the sidecar container?	`initContainers`, name `istio-proxy`
Which non-pod workloads exist?	`kubectl get workloadentry -A`

Ingress gateway on IPv6

Expose Bookinfo through the ingress gateway over IPv6

Bookinfo through the ingress gateway over IPv6: the way of a request



1, 2 The caller needs only IPv6 and the fixed address of the gateway. The firewall lets TCP 80 and 443 in.
3 The address lies in the /64 of one node. Only the MetalLB speaker of that node announces it, the traffic arrives there and goes on to the gateway pod (80 becomes 8080).
4, 5 Envoy in the gateway pod picks the route from the Gateway and the VirtualService and passes the request to the sidecar of productpage, which hands it to its app. The app calls details and reviews (reviews calls ratings).
Every pod has a sidecar (2/2), so every call between the pods goes through two sidecars (mTLS).
Limit: if the node with the /64 is down, the address is gone, even if the gateway pod runs on another node. Dashed grey line: configuration (through istiod), not traffic.

Background

Your lab has an Istio ingress gateway in the namespace `istio-ingress`. It is a normal Kubernetes Service of type `LoadBalancer`, IPv6 single stack. Two things make it reachable from other clusters and from outside the lab:

Part	What it does
Fixed address	Every lab server owns a whole `/64` network, routed to it by the provider. MetalLB hands out one fixed address from that `/64` to the gateway (`...::2:1`). No floating IP and no cloud load balancer is needed.
Cloud firewall	Allows TCP 80 and 443 from the internet. Everything else (SSH, API server, kubelet) stays closed.

In this exercise you publish the demo application Bookinfo through this gateway and call it from different places.

All commands run on your bastion host, in cluster A. The sample files come from the Istio release folder (`istio-1.29.9/samples`).

Step 1: Look at the gateway Service

```
kubectl get svc istio-ingressgateway -n istio-ingress
```

Expected output (addresses differ in your lab):

```
NAME                   TYPE           CLUSTER-IP          EXTERNAL-IP               PORT(S)
istio-ingressgateway   LoadBalancer   fd00:10:96::c:15b   2a01:4f8:c016:dab1::2:1
15021:31361/TCP,80:32665/TCP,443:31613/TCP
```

The EXTERNAL-IP is an IPv6 address. Check the address family and the target ports:

```
kubectl get svc istio-ingressgateway -n istio-ingress -o jsonpath='{.spec.ipFamilyPolicy} {.spec.ipFamilies}{"\n"}'
kubectl get svc istio-ingressgateway -n istio-ingress -o jsonpath='{.spec.ports[*].targetPort}{"\n"}'
```

```
SingleStack ["IPv6"]
15021 8080 8443
```

Port 80 of the Service leads to port **8080** of the gateway pod. The gateway pod does not run as root, so Envoy cannot listen on port 80 inside the pod. Keep this in mind, it matters in step 6.

Step 2: Deploy Bookinfo

```
kubectl label namespace default istio-injection=enabled --overwrite
kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
kubectl wait --for=condition=Ready pod --all -n default --timeout=300s
```

Every pod shows `2/2` (application and sidecar):

```
kubectl get pods -n default
```

Step 3: Publish it with a Gateway and a VirtualService

```
kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml
kubectl get gateway,virtualservice -n default
```

```
NAME                                          AGE
gateway.networking.istio.io/bookinfo-gateway   5s

NAME                                         GATEWAYS               HOSTS   AGE
virtualservice.networking.istio.io/bookinfo   ["bookinfo-gateway"]   ["*"]   5s
```

Step 4: Call Bookinfo

```
export GATEWAY=$(kubectl get svc istio-ingressgateway -n istio-ingress -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo $GATEWAY
curl -6 -s -o /dev/null -w "HTTP %{http_code}\n" "http://[$GATEWAY]/productpage"
```

```
HTTP 200
```

An IPv6 address in a URL needs square brackets. Without them `curl` reads the colons as a port.

Look at the content, the page shows the reviews and ratings that come from other pods:

```
curl -6 -s "http://[$GATEWAY]/productpage" | grep -E "<title>|Book Reviews|Reviewer"
```

Step 5: See what the gateway listens on

```
export GWPOD=$(kubectl get pod -n istio-ingress -l istio=ingressgateway -o jsonpath='{.items[0].metadata.name}')
istioctl proxy-config listener $GWPOD.istio-ingress
```

```
ADDRESSES PORT  MATCH DESTINATION
::        8080  ALL   Route: http.8080
::        15021 ALL   Inline Route: /healthz/ready*
::        15090 ALL   Inline Route: /stats/prometheus*
```

The addresses are `::`, not `0.0.0.0`. Envoy in an IPv6 single stack cluster listens on IPv6 only. The listener for your Gateway resource is on 8080: the `Gateway` of the sample names port 8080, the port of the gateway pod, and the Service port 80 leads there (step 1).

Step 6: Call it from other places

The gateway address is reachable from everywhere that has IPv6 connectivity to your lab:

Caller	Command	Expected
A node of the other cluster (cluster B)	`curl -6 -s -o /dev/null -w "HTTP %{http_code}\n" "http://[$GATEWAY]/productpage"`	HTTP 200
A host outside the lab with IPv6 (for example the trainer's machine)	the same command	HTTP 200

A host with IPv4 only	cannot work, there is no IPv4 address anywhere in the lab	no connection

If the call ends immediately with `Connection refused` although the Service has an endpoint, check the `targetPort` of the Service (step 1). With `targetPort` 80 the traffic arrives at a port where nothing listens. A quick test:

```
kubectl get endpointslice -n istio-ingress
```

The endpoint list is fine in that case, only the port is wrong. That is why this problem is easy to overlook.

Limits to know

- The fixed address belongs to the `/64` of **one** node. If this node is down, the address is gone, even if the gateway pod runs on another node.
- `externalTrafficPolicy: Cluster` (the default) hides the real client address from the gateway (source NAT).

Clean up

Keep Bookinfo, the next exercises use it. If you want to remove it:

```
kubectl delete -f samples/bookinfo/networking/bookinfo-gateway.yaml
kubectl delete -f samples/bookinfo/platform/kube/bookinfo.yaml
```

Two clusters, one mesh

Check the link between two clusters: `istioctl remote-clusters`

Background

Two clusters form one mesh when each `istiod` can read the services and endpoints of the other cluster. For this, every cluster holds a **remote secret**: a small kubeconfig that gives its `istiod` read access to the API server of the other cluster.

If a remote secret is missing or broken, nothing fails loudly. The other cluster is simply not part of the mesh, and traffic stays local. The command that shows this is:

```
istioctl remote-clusters
```

In this exercise you read the output, break the link on purpose, see what happens, and repair it.

All commands run on your bastion host. Replace the contexts with the ones of your two clusters:

```
export CTX_A=<context of cluster A>
export CTX_B=<context of cluster B>
```

The cluster names in the outputs below (`cluster-h6` , `cluster-h6b`) are the names in our test lab. Yours may differ.

Step 1: Prepare a test service in both clusters

`helloworld` runs in version v1 only in cluster A and in version v2 only in cluster B. Cluster A also gets a client pod (`sleep`). The sample files come from the Istio release folder (`istio-1.29.9/samples`).

```
for CTX in $CTX_A $CTX_B; do
  kubectl --context $CTX create namespace helloworld
  kubectl --context $CTX label namespace helloworld istio-injection=enabled
  kubectl --context $CTX apply -n helloworld -f samples/helloworld/helloworld.yaml -l service=helloworld
done
kubectl --context $CTX_A apply -n helloworld -f samples/helloworld/helloworld.yaml -l version=v1
kubectl --context $CTX_B apply -n helloworld -f samples/helloworld/helloworld.yaml -l version=v2
kubectl --context $CTX_A apply -n helloworld -f samples/sleep/sleep.yaml
kubectl --context $CTX_A wait -n helloworld --for=condition=Ready pod --all --timeout=180s
kubectl --context $CTX_B wait -n helloworld --for=condition=Ready pod --all --timeout=180s
```

Step 2: Look at the link

```
istioctl --context $CTX_A remote-clusters
```

```
NAME            SECRET                                          STATUS     ISTIOD
cluster-h6                                                      synced     istiod-7b66fbd84c-g682b
cluster-h6b     istio-system/istio-remote-secret-cluster-h6b    synced     istiod-7b66fbd84c-g682b
```

Read it column by column:

Column	Meaning
NAME	a cluster this `istiod` knows

SECRET	the remote secret it uses to reach that cluster. The first line is the local cluster and has no secret.
STATUS	`synced` means `istiod` has read the registry of that cluster
ISTIOD	the `istiod` pod that reports this

Now look from cluster B:

```
istioctl --context $CTX_B remote-clusters
```

Each cluster needs a secret for the **other** one. Cluster A holds the secret for B, and cluster B holds the secret for A. Both lines must say `synced`.

Step 3: See the effect on traffic

The client in cluster A calls `helloworld` 40 times:

```
kubectl --context $CTX_A exec -n helloworld deploy/sleep -c sleep -- sh -c 'for i in $(seq 1 40); do curl -s
http://helloworld.helloworld:5000/hello; done | grep -o "version: v[12]" | sort | uniq -c'
```

```
     22 version: v1
     18 version: v2
```

Answers come from both clusters. The client sidecar knows an endpoint in each cluster:

```
istioctl --context $CTX_A proxy-config endpoints deploy/sleep.helloworld --cluster
"outbound|5000||helloworld.helloworld.svc.cluster.local"
```

```
ENDPOINT                            STATUS      OUTLIER CHECK     CLUSTER
2a01:4f8:c016:17b5::1:9133:5000     HEALTHY     OK
outbound|5000||helloworld.helloworld.svc.cluster.local
2a01:4f8:c016:a758::1:a845:5000     HEALTHY     OK
outbound|5000||helloworld.helloworld.svc.cluster.local
```

Two addresses from two different networks: one pod in each cluster.

Step 4: Break the link

Remove the secret in cluster A that gives it access to cluster B. Use the secret name from the `SECRET` column of step 2:

```
kubectl --context $CTX_A -n istio-system delete secret istio-remote-secret-cluster-h6b
```

Wait ten seconds and look again:

```
istioctl --context $CTX_A remote-clusters
```

```
NAME           SECRET     STATUS     ISTIOD
cluster-h6                synced     istiod-7b66fbd84c-g682b
```

The line for cluster B is gone. There is no error message anywhere. Repeat the calls of step 3:

```
     40 version: v1
```

All 40 answers come from v1 and the client knows only one endpoint. Cluster B is not part of the mesh any more.

Take-away: If traffic does not reach the other cluster and there is no error, run `istioctl remote-clusters` first. A missing line (or a status other than `synced`) shows that the link is the problem.

Step 5: Repair the link

A remote secret for cluster B is created **in cluster B** (it needs the address and a reader account of that cluster) and applied **in cluster A**:

```
istioctl --context $CTX_B create-remote-secret --name=cluster-h6b | kubectl --context $CTX_A apply -f -
```

```
secret/istio-remote-secret-cluster-h6b created
```

The `--name` is the cluster name of B (the first column of step 2). After ten seconds:

```
istioctl --context $CTX_A remote-clusters
```

Both lines are back and say `synced`. Repeat the calls of step 3: answers from v1 and v2 again (for example 23 and 17).

Clean up

```
kubectl --context $CTX_A delete namespace helloworld
kubectl --context $CTX_B delete namespace helloworld
```

Summary

Question	Command
Does this `istiod` see the other cluster?	`istioctl remote-clusters`
Which proxies are connected to `istiod`?	`istioctl proxy-status`
Which endpoints does a sidecar know?	`istioctl proxy-config endpoints <pod>`
How do I join a cluster?	`istioctl create-remote-secret` in the other cluster, applied here

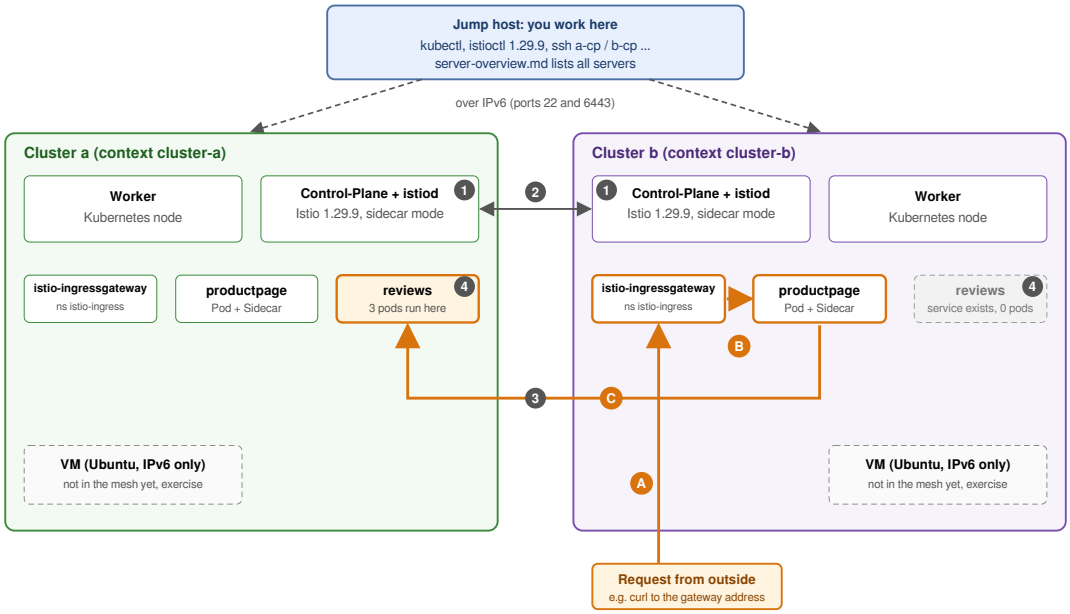
Keep the difference in mind: `remote-clusters` shows the link between the **control planes**, `proxy-status` shows which **proxies** are connected to each `istiod`.

Answer requests with pods from the other cluster

Background

In the previous exercise you checked the link between two clusters. Now you see what the link is good for: a request that enters cluster B can be answered by pods that run in cluster A, without a gateway between the clusters. The pods talk directly over IPv6, protected by mTLS.

Your lab: two clusters, one mesh. How cluster b answers a request with pods from cluster a



The path of a request (orange)
A The request arrives at the istio-ingressgateway of cluster b.
B The istio-ingressgateway passes it to productpage in cluster b.
C productpage calls reviews. No reviews pod runs in cluster b, so the pods in cluster a answer.

What it takes (grey numbers in the picture)
1 Same root CA: each side trusts the certificates of the other. **2** Remote secret: istiod reads the API of the other cluster.
3 Pods reach each other directly (IPv6, Calico without overlay, the firewall allows only your lab).
4 The service reviews exists in both clusters (same name and namespace). The pods only in one, in b the deployment may be at 0.

The orange arrows show the path of the request you follow in this exercise. The grey numbers are the four parts in the table below.

What makes this work:

Part	Where to see it
Both clusters trust the same root CA	step 1
Each `istiod` can read the other API server (remote secret)	`istioctl remote-clusters`, status `synced`
The pods of both clusters can reach each other (routed IPv6 network, same Istio `network`)	step 2

The service exists in both clusters (same name, same namespace)	Bookinfo from the exercise "Expose Bookinfo through the ingress gateway over IPv6"

Prerequisite: Bookinfo runs in **both** clusters and is published through both gateways (do the ingress gateway exercise in cluster A and in cluster B).

```
export CTX_A=<context of cluster A>
export CTX_B=<context of cluster B>
export GATEWAY_B=<external address of the ingress gateway in cluster B>
```

Step 1: Compare the root certificates

```
for CTX in $CTX_A $CTX_B; do
  kubectl --context $CTX get cm istio-ca-root-cert -n default -o jsonpath='{.data.root-cert\.pem}' | openssl x509 -
noout -fingerprint -sha256
done
```

Both lines must show the same fingerprint. If they differ, the clusters do not trust each other's certificates and cross-cluster traffic fails.

Step 2: Look at the endpoints of `reviews`

The Envoy of `productpage` in cluster B knows all endpoints of the service `reviews`:

```
export PP_B=$(kubectl --context $CTX_B get pod -l app=productpage -o jsonpath='{.items[0].metadata.name}')
istioctl --context $CTX_B proxy-config endpoints $PP_B --cluster "outbound|9080||reviews.default.svc.cluster.local"
```

Expected output: six endpoints, three from the network of each cluster (addresses differ in your lab):

```
ENDPOINT                          STATUS      OUTLIER CHECK     CLUSTER
2a01:4f8:c016:a758::1:9110:9080   HEALTHY     OK                outbound|9080||reviews.default.svc.cluster.local
2a01:4f8:c016:a758::1:9111:9080   HEALTHY     OK                outbound|9080||reviews.default.svc.cluster.local
2a01:4f8:c016:a758::1:9112:9080   HEALTHY     OK                outbound|9080||reviews.default.svc.cluster.local
2a01:4f8:c016:aa79::1:a84e:9080   HEALTHY     OK                outbound|9080||reviews.default.svc.cluster.local
2a01:4f8:c016:aa79::1:a853:9080   HEALTHY     OK                outbound|9080||reviews.default.svc.cluster.local
2a01:4f8:c016:aa79::1:a854:9080   HEALTHY     OK                outbound|9080||reviews.default.svc.cluster.local
```

The first three belong to cluster A, the last three to cluster B. Compare with the pod addresses:

```
kubectl --context $CTX_A get pods -l app=reviews -o wide
kubectl --context $CTX_B get pods -l app=reviews -o wide
```

Step 3: Take away the local pods

Scale `reviews` to zero in cluster B:

```
for D in reviews-v1 reviews-v2 reviews-v3; do kubectl --context $CTX_B scale deploy $D --replicas=0; done
kubectl --context $CTX_B wait --for=delete pod -l app=reviews --timeout=90s
kubectl --context $CTX_B get pods -l app=reviews
```

```
No resources found in default namespace.
```

Wait a few seconds, then look at the endpoints again:

```
istioctl --context $CTX_B proxy-config endpoints $PP_B --cluster "outbound|9080||reviews.default.svc.cluster.local"
```

Only the three endpoints of cluster A are left.

Step 4: Call cluster B from outside

```
for i in 1 2 3 4 5 6; do
  curl -6 -s "http://[$GATEWAY_B]/productpage" | grep -c Reviewer1
done
```

```
1
1
1
1
1
1
```

Every answer contains the reviews, although cluster B has no `reviews` pod. The page of cluster B was built with pods of cluster A.

Step 5: Repair

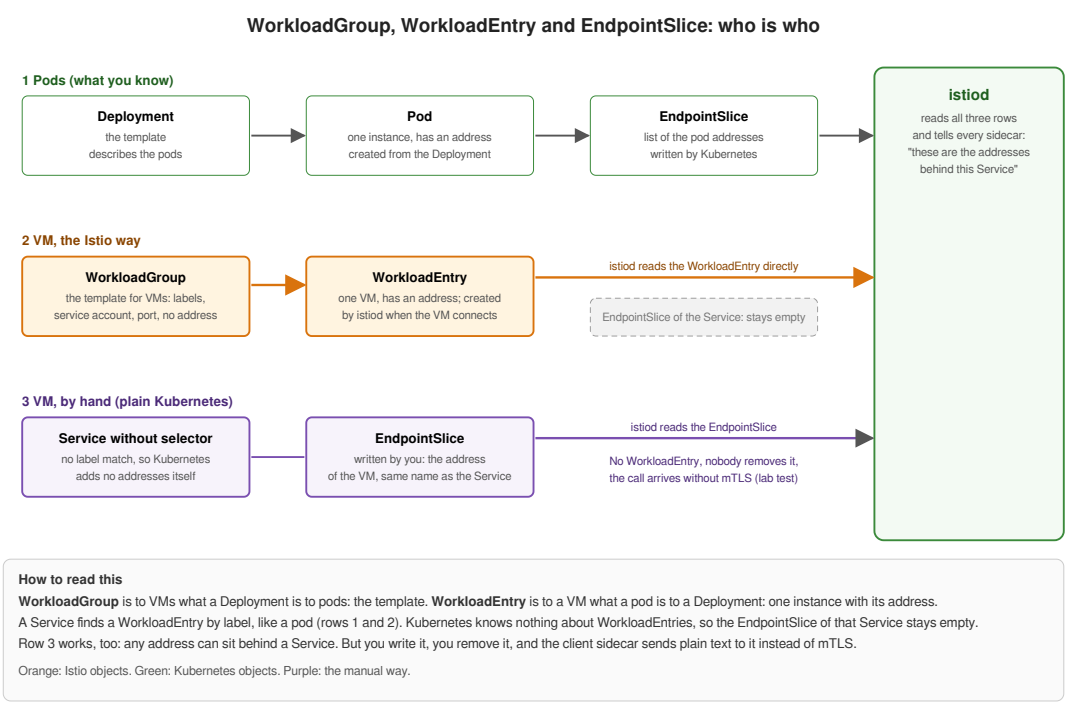
```
for D in reviews-v1 reviews-v2 reviews-v3; do kubectl --context $CTX_B scale deploy $D --replicas=1; done
kubectl --context $CTX_B get pods -l app=reviews
```

Questions

- What would you see in step 2 if the remote secret of cluster B were missing in cluster A?
- The root CA must be the same in both clusters. What happens to the connection between pods of the two clusters if you replace the CA in only one cluster?

Virtual machines in the mesh

WorkloadGroup, WorkloadEntry and EndpointSlice: who is who



Background

For pods, Kubernetes does the bookkeeping: a Deployment describes the pods, the pods have addresses, and Kubernetes writes the addresses of the pods behind a Service into an `EndpointSlice`. `istiod` reads these and tells every sidecar where a Service lives.

A virtual machine is not a pod. Kubernetes does not know it. Istio adds two objects so that a VM can be a part of the mesh like a pod:

Object	Compare to	What it is	Who creates it
`WorkloadGroup`	Deployment	The template for VMs: labels, service account, port, network. It has no address.	You
`WorkloadEntry`	Pod	One VM with its address, labels and service account.	`istiod`, when the VM connects (auto-registration). You can write one by hand, too.
`EndpointSlice`	-	Plain Kubernetes: the list of addresses behind a Service.	Kubernetes, for pods that a Service selects. For anything else: you.

A `Service` finds a `WorkloadEntry` by label, exactly as it finds a pod. `istiod` reads the `WorkloadEntry` directly. Kubernetes does not turn it into an `EndpointSlice`, so the `EndpointSlice` of that Service stays empty. This is normal.

Two ways to put a VM behind a Service

	`WorkloadEntry` (Istio way)	Own `EndpointSlice` (manual way)
Created by	`istiod`, when the VM connects	you

Removed	by `istiod`, shortly after the VM has gone (in our test: within 10 seconds after we stopped the sidecar)	never, you do it (in our test it was still there)
Address	the one the sidecar reports (`ISTIO_SVC_IP`, if set)	the one you write
Service	with selector (label of the `WorkloadGroup`)	without selector, `EndpointSlice` with the label `kubernetes.io/service-name`
Call from a pod	mTLS (we saw `x-forwarded-client-cert`)	in our test: plain text, no `x-forwarded-client-cert`

Step 1: Look at the objects

Do this after you have finished the exercise Add a virtual machine to the mesh (the VM is in the mesh, the pod `client` runs).

```
export CTX_A=cluster-a
kubectl --context $CTX_A -n vm get workloadgroup,workloadentry
```

```
NAME                                                AGE
workloadgroup.networking.istio.io/diameter-adapter   29s

NAME                                                                        AGE   ADDRESS
workloadentry.networking.istio.io/diameter-adapter-2a01-4f8-c013-541--1-net1   23s   2a01:4f8:c013:541::1
```

The name of the `WorkloadEntry` is the name of the `WorkloadGroup` plus the address of the VM. Now the `EndpointSlice` of the same Service:

```
kubectl --context $CTX_A -n vm get endpointslice
```

```
NAME                     ADDRESSTYPE   PORTS     ENDPOINTS   AGE
diameter-adapter-jmpcj   IPv6          <unset>   <unset>     29s
```

It is empty. Still the call from the pod works, because `istiod` knows the address from the `WorkloadEntry` .

Step 2: Put the same VM behind a second Service by hand

A `Service` without selector and an `EndpointSlice` with the address of the VM. `VM_IP` is the IPv6 address of your VM (see `ssh a-vm1 ip -6 -br a show eth0`).

```
export VM_IP=<IPv6 address of the VM>
```

```
## vi 03-manual-endpointslice.yml
apiVersion: v1
kind: Service
metadata:
  name: diameter-manual
  namespace: vm
spec:
  ipFamilyPolicy: SingleStack
  ipFamilies: [IPv6]
  ports:
    - name: http
      port: 8000
      targetPort: 8000
---
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: diameter-manual-1
  namespace: vm
  labels:
    kubernetes.io/service-name: diameter-manual
addressType: IPv6
ports:
  - name: http
    port: 8000
    protocol: TCP
endpoints:
  - addresses:
      - "<IPv6 address of the VM>"
```

Put your address into the file (`sed -i "s/<IPv6 address of the VM>/$VM_IP/" 03-manual-endpointslice.yml`), then:

```
kubectl --context $CTX_A apply -f 03-manual-endpointslice.yml
```

Step 3: Compare the two calls

Both Services lead to the same VM:

```
istioctl --context $CTX_A proxy-config endpoint deploy/client | grep -E "ENDPOINT|diameter"
```

```
ENDPOINT                     STATUS      OUTLIER CHECK     CLUSTER
2a01:4f8:c013:541::1:8000    HEALTHY     OK                outbound|8000||diameter-adapter.vm.svc.cluster.local
2a01:4f8:c013:541::1:8000    HEALTHY     OK                outbound|8000||diameter-manual.vm.svc.cluster.local
```

Call both and look for the header `x-forwarded-client-cert`:

```
kubectl --context $CTX_A exec deploy/client -c curl -- curl -s -m 8 http://diameter-adapter.vm.svc.cluster.local:8000/
| grep -E "VM diameter|x-forwarded-client-cert"
kubectl --context $CTX_A exec deploy/client -c curl -- curl -s -m 8 http://diameter-manual.vm.svc.cluster.local:8000/
| grep -E "VM diameter|x-forwarded-client-cert"
```

The first call shows the line with `x-forwarded-client-cert` (mTLS, both identities). The second call shows only `VM diameter-adapter (IPv6) ...`: the request arrived, but without mTLS.

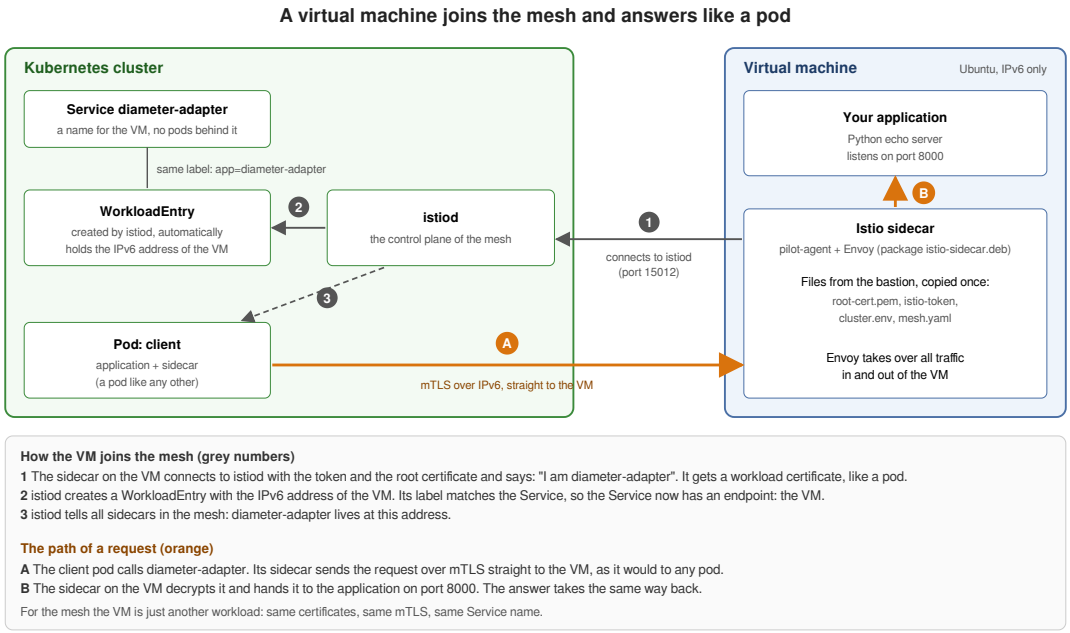
What to take with you

- `WorkloadGroup` is the template, `WorkloadEntry` is the instance. You write the first, `istiod` writes the second.
- The `EndpointSlice` of a Service with a `WorkloadEntry` stays empty. Do not look for the VM there.
- A hand-made `EndpointSlice` is plain Kubernetes. It works, but it does not give the VM the properties of a mesh workload (automatic registration and removal, mTLS from the client). If you see both objects for a VM in a real environment, ask why the `EndpointSlice` exists.

Clean up

```
kubectl --context $CTX_A -n vm delete -f 03-manual-endpointslice.yml
```

Add a virtual machine to the mesh (IPv6 only, VM without IPv4)



How the VM joins the mesh (grey numbers)
1 The sidecar on the VM connects to istiod with the token and the root certificate and says: "I am diameter-adapter". It gets a workload certificate, like a pod.
2 istiod creates a WorkloadEntry with the IPv6 address of the VM. Its label matches the Service, so the Service now has an endpoint: the VM.
3 istiod tells all sidecars in the mesh: diameter-adapter lives at this address.

The path of a request (orange)
A The client pod calls diameter-adapter. Its sidecar sends the request over mTLS straight to the VM, as it would to any pod.
B The sidecar on the VM decrypts it and hands it to the application on port 8000. The answer takes the same way back.
For the mesh the VM is just another workload: same certificates, same mTLS, same Service name.

> ***Attention:*** *When you use IPv6 only on the VMs (Istio hybrid), be sure that the IPv4 address on the VM is disabled. That is the best and easiest way. Otherwise you need to fiddle around (dual stack switches,* `ISTIO_SVC_IP` *, health* `probe` *workarounds).*

Background

A virtual machine joins the mesh with the Istio sidecar package (`istio-sidecar.deb`). The sidecar connects to `istiod` and gets a workload certificate. The VM is **not** a pod and does not show up in `kubectl get pods` . `istiod` creates a `WorkloadEntry` for it automatically (auto-registration), and it appears in `istioctl proxy-status` like the sidecar of a pod. From then on a pod calls the VM through a normal Kubernetes `Service` , with mTLS.

What `WorkloadGroup`, `WorkloadEntry` and `EndpointSlice` are and how they differ: who is who.

Kubernetes, Istio and the VM run as IPv6 single stack. The lab VM has an IPv6 address and **no IPv4 address** (the private address `100.65.x.x` that the cloud gives it is switched off in the VM). Then Istio needs no dual stack switches and no extra settings on the VM. You can check it on the VM:

```
ssh a-vm1 "ip -4 -br addr show eth0; ip -6 -br addr show eth0"
```

The first command prints nothing, the second the IPv6 address. If your VM has an IPv4 address, switch it off first. The file tells the VM not to ask for an IPv4 address by DHCP:

```
ssh a-vm1 "printf 'network:\n  version: 2\n  ethernets:\n    eth0:\n      dhcp4: false\n' > /etc/netplan/99-vm-no-ipv4.yaml && chmod 600 /etc/netplan/99-vm-no-ipv4.yaml && netplan apply"
```

(Or follow the optional page with IPv4 and IPv6 (dual stack).)

Part	Name in this exercise
VM	`a-vm1` (reach it with `ssh a-vm1`)
Namespace, service account, workload group, service	`vm`, `diameter-adapter`
Application on the VM	a small Python echo server on port 8000 (`vm-echo-server.py`)
Address of `istiod` for the VM	a fixed `LoadBalancer` address (`istiod-lb`, port 15012)

How to do this exercise

We go through the steps one by one. For every step the trainer explains what happens and why. You do **not** type everything by hand: the script `vm-in-the-mesh.sh` (in the folder `virtual-machines` on your bastion host) runs the steps for you.

```
cd virtual-machines
./vm-in-the-mesh.sh
```

For every step the script prints the commands first and waits:

```
... to continue press ENTER
```

Press ENTER and the script runs exactly the commands you see. You can copy any of them and type it yourself. The run itself takes about one minute, plus the time for the explanations.

The steps

Step 1: Settings

The script sets the contexts of both clusters, the name of the VM and the IPv6 address of the VM. It also reads the IPv4 address of the VM and stops if there is one.

```
export CTX_A=cluster-a
export CTX_B=cluster-b
export VM=a-vm1
export VM_IP=$(ssh $VM "ip -6 -o addr show dev eth0 scope global" | awk '{print $4}' | cut -d/ -f1 | head -1)
export VM_IP4=$(ssh $VM "ip -4 -o addr show dev eth0 scope global" | awk '{print $4}' | head -1)
```

Step 2: Make `istiod` ready for VMs

The VM is outside the cluster and must reach `istiod` from outside. The script does two things:

- It switches on auto-registration (`PILOT_ENABLE_WORKLOAD_ENTRY_AUTOREGISTRATION`) and the DNS proxy with `helm upgrade istiod ... --reuse-values` .
- It creates the Service `istiod-lb` of type `LoadBalancer` with a fixed IPv6 address (MetalLB), port 15012.

If your lab is already prepared, the script says so and skips the step.

```
kubectl --context $CTX_A -n istio-system get svc istiod-lb
```

```
NAME         TYPE           CLUSTER-IP          EXTERNAL-IP               PORT(S)
istiod-lb    LoadBalancer   fd00:10:96::d:ec63  2a01:4f8:1c17:5cbc::2:2   15012:32132/TCP
```

The VM uses the EXTERNAL-IP. The cluster IP (`fd00:...`) exists only inside the cluster.

Step 3: Create the objects for the VM

Namespace `vm` , service account `diameter-adapter` and the Service `diameter-adapter` in **both** clusters (so that cluster b finds the VM, too). The `WorkloadGroup` (the template for the VM) only in cluster a. The Service finds the VM by the label `app=diameter-adapter` . The script writes the two files `01-vm-objects.yml` and `02-workloadgroup.yml` and shows their content.

```
kubectl --context $CTX_A apply -f 01-vm-objects.yml -f 02-workloadgroup.yml
kubectl --context $CTX_B apply -f 01-vm-objects.yml
```

Step 4: Create the files for the VM

The command is `istioctl x workload entry configure` (`x` is short for `experimental`). It reads the `WorkloadGroup` and the mesh configuration of the cluster and writes everything the VM needs into a folder. Then the script copies the folder to the VM.

```
istioctl --context $CTX_A x workload entry configure -f 02-workloadgroup.yml -o vmfiles --clusterID cluster-a --
ingressIP $ISTIOD_ADDR --autoregister
ls vmfiles
scp -q -r vmfiles vm-echo-server.py $VM:/root/
```

Option	Meaning
`-f 02-workloadgroup.yml`	the `WorkloadGroup` of the VM (name, namespace, labels, service account, ports, network)
`-o vmfiles`	the folder for the files
`--clusterID cluster-a`	the name of the cluster the VM registers with
`--ingressIP $ISTIOD_ADDR`	the address of `istiod` that the VM can reach (our `istiod-lb`, see step 2)
`--autoregister`	`istiod` creates the `WorkloadEntry` when the VM connects. Without it the `WorkloadEntry` is missing

The command writes five files:

```
cluster.env  hosts  istio-token  mesh.yaml  root-cert.pem
```

File	Goes to (on the VM)	What is in it
`cluster.env`	`/var/lib/istio/envoy/cluster.env`	One `KEY='value'` line per setting. `istio-start.sh` reads it and every line becomes an environment variable of the sidecar. See below
`mesh.yaml`	`/etc/istio/config/mesh`	The mesh configuration for the agent: `discoveryAddress: istiod.istio-system.svc:15012`, `meshId`, and under `proxyMetadata` the same entries as in `cluster.env`
`hosts`	appended to `/etc/hosts`	One line: `<ISTIOD_ADDR> istiod.istio-system.svc`, so the VM finds `istiod` by name
`root-cert.pem`	`/etc/certs/root-cert.pem`	The root certificate of the mesh (in our lab `Lab Root CA tln1`), so the VM trusts `istiod` and the other workloads
`istio-token`	`/var/run/secrets/tokens/istio-token`	A token (JWT) of the service account `diameter-adapter` in the namespace `vm`. The agent uses it once to get its workload certificate. **Secret**, valid for a short time (one hour by default). Do not print it and do not commit it

This is the `cluster.env` of our lab (lines in the order `istioctl` writes them). There is no dual stack line, because `istiod` has no `ISTIO_DUAL_STACK` setting:

```
CANONICAL_REVISION='latest'
CANONICAL_SERVICE='diameter-adapter'
ISTIO_INBOUND_PORTS='8000'
ISTIO_LOCAL_EXCLUDE_PORTS='22,15090,15021,15020'
ISTIO_METAJSON_LABELS='{"app":"diameter-adapter",...}'
ISTIO_META_AUTO_REGISTER_GROUP='diameter-adapter'
ISTIO_META_CLUSTER_ID='cluster-a'
ISTIO_META_DNS_AUTO_ALLOCATE='true'
ISTIO_META_DNS_CAPTURE='true'
ISTIO_META_MESH_ID='mesh-tln1'
ISTIO_META_NETWORK='net1'
ISTIO_META_POD_PORTS='[{"name":"http","containerPort":8000,"protocol":""}]'
ISTIO_META_WORKLOAD_NAME='diameter-adapter'
ISTIO_NAMESPACE='vm'
ISTIO_SERVICE='diameter-adapter.vm'
ISTIO_SERVICE_CIDR='*'
POD_NAMESPACE='vm'
SERVICE_ACCOUNT='diameter-adapter'
TRUST_DOMAIN='cluster.local'
```

Where the lines come from:

- **From the `WorkloadGroup` :** `ISTIO_NAMESPACE` , `POD_NAMESPACE` , `SERVICE_ACCOUNT` , `ISTIO_SERVICE` , `ISTIO_INBOUND_PORTS` (the ports of the `WorkloadGroup`), `ISTIO_METAJSON_LABELS` (its labels), `ISTIO_META_NETWORK` (its network), `ISTIO_META_POD_PORTS` .
- **From the options:** `ISTIO_META_CLUSTER_ID` (`--clusterID`), `ISTIO_META_AUTO_REGISTER_GROUP` (`--autoregister`).

- **From the mesh configuration of the cluster (`proxyMetadata` in the ConfigMap `istio` in `istio-system`):** every entry there, for example `ISTIO_META_DNS_CAPTURE` and `ISTIO_META_DNS_AUTO_ALLOCATE` . They appear in `mesh.yaml` and in `cluster.env` . (With the dual stack setting in `istiod` , `ISTIO_DUAL_STACK` would appear here, too: see the optional dual stack page.)
- **Fixed by `istioctl` :** `ISTIO_LOCAL_EXCLUDE_PORTS='22,15090,15021,15020'` . These ports are not redirected to Envoy: SSH and the three ports of the sidecar itself.

The script does **not** change `cluster.env` : for a VM with IPv6 only, what `istioctl` wrote is enough.

Step 5: Install the sidecar on the VM and start it

The script logs in to the VM and does this: install `istio-sidecar.deb` (the download goes over IPv6), copy the files to the places the sidecar reads them from, start the sidecar and the echo server.

Step 6: Is the VM in the mesh?

```
istioctl --context $CTX_A proxy-status | grep -E "NAME|vm"
kubectl --context $CTX_A get workloadentry -n vm
```

```
NAME                 CLUSTER     ISTIOD                     VERSION   SUBSCRIBED TYPES
v6-tln1-a-vm1.vm     cluster-a   istiod-59c4f47fc9-b2vwb    1.29.9    5 (CDS,LDS,EDS,RDS,NDS)

NAME                                        AGE   ADDRESS
diameter-adapter-2a01-4f8-c013-541--1-net1   10s   2a01:4f8:c013:541::1
```

The VM is connected to `istiod` , and the `WorkloadEntry` holds the IPv6 address of the VM. This does not show yet that traffic works.

Step 7: Test in three directions

From a pod to the VM:

```
kubectl --context $CTX_A exec deploy/client -c curl -- curl -s -m 8 -w "\nHTTP %{http_code}\n" http://diameter-
adapter.vm.svc.cluster.local:8000/
```

```
VM diameter-adapter (IPv6), client=::6
x-forwarded-client-cert: By=spiffe://cluster.local/ns/vm/sa/diameter-
adapter;Hash=...;URI=spiffe://cluster.local/ns/default/sa/default
HTTP 200
```

The header `x-forwarded-client-cert` shows the identities of both sides: mTLS between the pod and the VM.

From the VM to a service in the cluster (the sidecar of the VM resolves the name, DNS proxy):

```
ssh $VM 'curl -s -m 8 -o /dev/null -w "HTTP %{http_code}\n"
http://productpage.default.svc.cluster.local:9080/productpage'
```

From a pod in **cluster b** to the VM (the VM is registered in cluster a, cluster b finds it through the mesh link):

```
kubectl --context $CTX_B exec deploy/client -c curl -- curl -s -m 8 -w "\nHTTP %{http_code}\n" http://diameter-
adapter.vm.svc.cluster.local:8000/
```

All three answer `HTTP 200` .

What to check on the VM

The sidecar sets iptables rules and Envoy listens on the IPv6 wildcard. On the VM (`ssh a-vm1`):

```
echo "IPv4 rules: $(iptables-save -t nat | grep -c ISTIO)  IPv6 rules: $(ip6tables-save -t nat | grep -c ISTIO)"
ss -lnt | grep -E ':1500[16] '
```

```
IPv4 rules: 23  IPv6 rules: 19
LISTEN 0      4096            [::]:15006         [::]:*
LISTEN 0      4096            [::]:15006         [::]:*
LISTEN 0      4096            [::]:15001         [::]:*
LISTEN 0      4096            [::]:15001         [::]:*
```

The tool always sets the IPv4 rules, and it sets the IPv6 rules because the first address of the VM is an IPv6 address. Nothing to configure. Every port shows twice in our lab; what matters is `[::]` , the IPv6 wildcard address.

If a VM has an IPv4 address, things change: the sidecar may register the IPv4 address (calls fail with `503`) and needs the dual stack switches. That is the topic of the optional page and of DUAL_STACK and ISTIO_DUAL_STACK.

Good to know

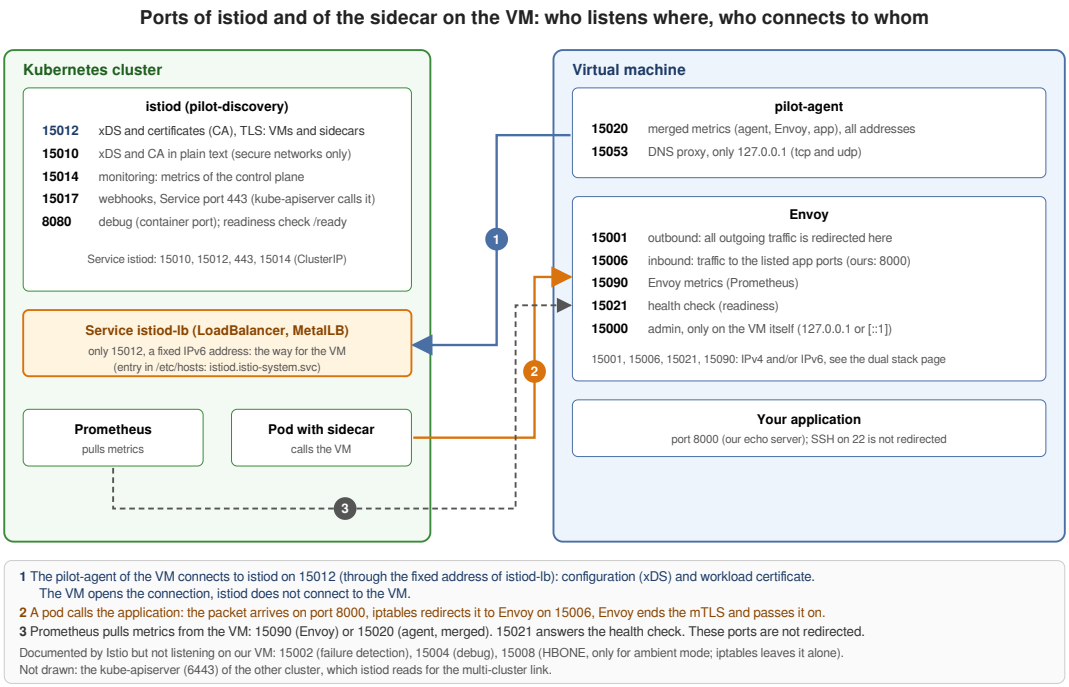
- The sidecar service is not enabled at boot (`systemctl is-enabled istio` shows `disabled`). After a reboot of the VM the sidecar does not start by itself.
- While the sidecar redirects IPv6 traffic to a port where Envoy does not listen, the outgoing IPv6 traffic of the VM is disturbed, too (a download with `curl` fails). SSH keeps working (port 22 is excluded).
- The files in `vmfiles` contain a token. Do not share them.

Clean up

```
./vm-in-the-mesh.sh cleanup
```

The script deletes the namespace `vm` in both clusters and the two test pods. On the VM it stops the sidecar and the echo server and removes the sidecar package, its files and the echo server. The VM is back in the state before the exercise. The settings of `istiod` stay.

Ports of istiod and of the sidecar on the VM



Background

Two programs matter for a VM in the mesh:

- **`istiod`** (the program is called `pilot-discovery`) runs in the cluster. It gives the sidecars their configuration (xDS) and signs the workload certificates (CA).
- **The sidecar** runs on the VM: **`pilot-agent`** (the agent, it talks to `istiod`) and **Envoy** (the proxy, it carries the traffic).

The tables list every port. The column "Istio docs" is from the Istio documentation (*Ports used by Istio*), the column "Measured" is what we saw on our lab (Istio 1.29.9, `ss -ltnp` on the VM, `kubectl` for `istiod`).

The sidecar on the VM

Port	Who listens	Istio docs	Measured on our VM	Who connects
`15000`	Envoy	Envoy admin port (commands, diagnostics), pod-internal only	`127.0.0.1`, or `[::1]` with `ISTIO_DUAL_STACK` or on a VM with IPv6 only	only the VM itself (the agent asks it if Envoy is ready; you with `curl`)
`15001`	Envoy	Envoy outbound	`0.0.0.0` and `[::]` (IPv6 only: `[::]`)	iptables redirects all outgoing TCP traffic of the VM here
`15006`	Envoy	Envoy inbound	`0.0.0.0` and `[::]` (IPv6 only: `[::]`)	iptables redirects incoming traffic for the listed application ports here (ours: `8000`)
`15020`	`pilot-agent`	merged Prometheus telemetry from agent, Envoy and application	all addresses (`*`), IPv4 and IPv6, with every setting	Prometheus

`15021`	Envoy	health checks	`0.0.0.0`; also `[::]` with `ISTIO_DUAL_STACK` (IPv6 only: `[::]`)	health checks, Prometheus
`15053`	`pilot-agent`	DNS port, if DNS capture is enabled, pod-internal only	`127.0.0.1`, tcp and udp	only the VM itself (the DNS proxy, switched on by `ISTIO_META_DNS_CAPTURE`)
`15090`	Envoy	Envoy Prometheus telemetry	`0.0.0.0`; also `[::]` with `ISTIO_DUAL_STACK` (IPv6 only: `[::]`)	Prometheus
`8000`	our application	-	`*` (the Python echo server)	other workloads (redirected to `15006`)
`22`	`sshd`	-	`0.0.0.0` and `[::]`	you (not redirected)

Listed by Istio, but **not listening** on our VM: `15002` (failure detection), `15004` (debug port), `15008` (HBONE tunnel, used in ambient mode; the iptables rules leave it alone in sidecar mode).

What iptables redirects

`istio-start.sh` runs `pilot-agent istio-iptables`. On our VM it sets up:

- **Outbound:** all outgoing TCP traffic goes to `15001`.
- **Inbound:** only the ports in `ISTIO_INBOUND_PORTS` go to `15006`. `istioctl` takes them from the ports of the `WorkloadGroup` (ours: `8000`). The rule on our VM: `-A ISTIO_INBOUND -p tcp --dport 8000 -j ISTIO_IN_REDIRECT`, and `15008` is left alone.
- **Not redirected:** every other port, so also `15020`, `15021`, `15090` and `22`. The line `ISTIO_LOCAL_EXCLUDE_PORTS='22,15090,15021,15020'` from `istioctl` is an exclusion list that only applies when *all* inbound ports are redirected (`*`). Ours are not.

If you list more application ports in the `WorkloadGroup`, they go to `15006`, too.

istiod in the cluster

Port	Name in the Deployment	Istio docs	Our lab	Who connects
`15012`	`tls-xds`	XDS and CA services, TLS and mTLS, recommended for production	in the `Service istiod` (named `https-dns`) and in `istiod-lb`	**the VM**, and the sidecars of the pods
`15010`	`grpc-xds`	XDS and CA services in plain text, only for secure networks	in the `Service istiod`; the VM does not use it	sidecars that are set up for it (we do not use it)
`15014`	`http-monitoring`	control plane monitoring	`--monitoringAddr=:15014`, in the `Service istiod`	Prometheus (metrics of `istiod`)
`15017`	`https-webhooks`	webhook container port, forwarded from `443`	the `Service istiod` maps `443` to `15017`	the `kube-apiserver` (sidecar injection, validation)
`8080`	`http-debug`	debug interface (deprecated, container port only)	container port only, not in a Service; the readiness probe of `istiod` asks `/ready` here	the kubelet

Our extra Service for VMs:

Service	Type	Port	Meaning
`istiod-lb`	`LoadBalancer` (MetalLB)	`15012` only	a **fixed IPv6 address** that the VM can reach. The `hosts` file of the VM maps `istiod.istio-system.svc` to it. The cluster IP of `istiod` exists only inside the cluster

Other ports in our lab: the gateway `istio-ingressgateway` has `15021` (status), `80` (to `8080`) and `443` (to `8443`). The `kube-apiserver` listens on `6443` (the multi-cluster link: `istiod` of one cluster reads the API of the other, with the remote secret).

What this means for a firewall

For one VM in the mesh the following must be allowed:

From	To	Port	For
VM	`istiod` (address of `istiod-lb`)	`15012/TCP`	configuration and certificate
other pods and VMs (workloads in the mesh)	VM	the application port (`8000`)	the traffic itself; the VM answers through Envoy
VM	pods and VMs of the mesh	the application ports of the other side	the outgoing traffic (the network is flat, pods are reached directly)
Prometheus	VM	`15090` (Envoy) or `15020` (agent), optional `15021`	metrics

you	VM	22/TCP	SSH (not part of Istio)

`istiod` does not open a connection to the VM. The VM connects, and the answers come back on the same connection. What the lab firewall of Hetzner allows is not part of this page.

Look at your VM

```
ssh $VM "ss -ltnpH | awk '{print \$4, \$NF}' | sed 's/users:(((/; s/,pid=[0-9]*,fd=[0-9]*))//' | sort -t: -k2 -n |
uniq"
ssh $VM "ss -lunpH | awk '{print \$4, \$NF}' | sed 's/users:(((/; s/,pid.*//' | sort -u"
ssh $VM "iptables-save -t nat | grep -E 'ISTIO_INBOUND|ISTIO_REDIRECT'"
kubectl --context $CTX_A -n istio-system get deploy istiod -o jsonpath='{range
.spec.template.spec.containers[0].ports[*]}{.name}={.containerPort} {end}'
kubectl --context $CTX_A -n istio-system get svc istiod istiod-lb
```

Which addresses `15000`, `15021` and `15090` use depends on the dual stack setting: Dual stack on the VM: what to set.

Debugging Istio

Logs and log levels

When something does not work, the logs of `istiod` and of the sidecar are the first place to look. This page shows where they are, how to raise the level for a short time and how to set it for good. Everything was measured in our lab (Istio 1.29.9).

Where the logs are

```
kubectl -n istio-system logs deploy/istiod --tail=3
kubectl logs deploy/productpage-v1 -c istio-proxy --tail=3
```

```
2026-11-06T18:30:47.047692Z     info    delta   EDS: PUSH request for node:istio-ingressgateway-78c9c8b488-
qd2rr.istio-ingress resources:20 ...
2026-11-06T17:03:11.791054Z     info    xdsproxy        connected to delta upstream XDS server: istiod.istio-
system.svc:15012        id=40
```

A line is: time, **level**, **scope**, message. The scope tells you which part of the program wrote it (`delta`, `ads`, `xdsproxy`, `envoy http`). The container `istio-proxy` holds the logs of the `pilot-agent` and of Envoy together. The iptables setup of the pod is in the container `istio-init`.

Default levels in our lab: `istiod` and `pilot-agent` write `info`, Envoy writes `warning`. Debug levels write many more lines, so raise a level only where you need it and reset it afterwards.

Raise the level for a short time

This changes the running process. There is no restart.

`istiod` : the levels belong to *scopes*. The format is `<scope>:<level>`, several scopes are separated by a comma. Levels: `none`, `error`, `warn`, `info`, `debug`.

```
istioctl admin log
istioctl admin log --level ads:debug
istioctl admin log --log-reset
```

The first command lists the scopes, here the first rows:

```
ACTIVE SCOPE           DESCRIPTION                                          LOG LEVEL   STACK TRACE LEVEL
ads                    ads debugging                                        info        none
adsc                   adsc debugging                                       info        none
analysis               Scope for configuration analysis runtime             info        none
```

After the second command the row of `ads` shows `debug`, after `--log-reset` it is `info` again. `ads` is the scope for the stream with which `istiod` sends the configuration to the proxies, see xDS in Ports of istiod and of the sidecar. `istioctl admin log` talks to `istiod` through ControlZ.

The sidecar (Envoy): the levels belong to *loggers*.

```
istioctl proxy-config log deploy/productpage-v1
istioctl proxy-config log deploy/productpage-v1 --level http:debug
istioctl proxy-config log deploy/productpage-v1 -r
```

The first command lists all loggers of Envoy, all on `warning`. The second sets one logger, `-r` resets all to the default (`warning`).

Try it: see how `istiod` pushes a change

Raise `ads`, change a harmless Istio resource and read the log:

```
istioctl admin log --level ads:debug
kubectl apply -f - <<EOF
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: dbg-test
  namespace: default
spec:
  host: productpage
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
EOF
kubectl -n istio-system logs deploy/istiod --since=1m | grep -E "ads|debug"
```

```
2026-11-06T18:31:05.456098Z     info    ads     Push debounce stable[20] 1 for config DestinationRule/default/dbg-
test: 100.264087ms since last change, 100.264017ms since last push, full=true
2026-11-06T18:31:05.456282Z     debug   ads     InitContext 2026-11-06T18:31:05Z/13 for push took 73.74µs
2026-11-06T18:31:05.456293Z     info    ads     XDS: Pushing Services:14 ConnectedEndpoints:7 Version:2026-11-
06T18:31:05Z/13
```

You see that `istiod` noticed the change of `DestinationRule/default/dbg-test` and pushed to 7 connected proxies. Clean up and reset the level:

```
kubectl delete destinationrule dbg-test -n default
istioctl admin log --log-reset
```

In our lab `istiod` pushes the configuration to the sidecars as *delta* updates (only the changes). The `info` lines of the scope `delta` show it, as in the example above.

Try it: see the requests of a sidecar

```
istioctl proxy-config log deploy/productpage-v1 --level http:debug
kubectl run curl-test --image=curlimages/curl:8.11.1 --restart=Never -- sleep 600
kubectl wait --for=condition=Ready pod/curl-test --timeout=120s
kubectl exec curl-test -c curl-test -- curl -s -o /dev/null -w "HTTP %{http_code}\n"
http://productpage:9080/productpage
kubectl logs deploy/productpage-v1 -c istio-proxy --since=1m | grep http
```

```
HTTP 200
2026-11-06T18:31:14.317186Z     debug   envoy http external/envoy/source/common/http/conn_manager_impl.cc:2019  [Tags:
"ConnectionId":"1146","StreamId":"4067863431051959824"] encoding headers via codec ...
2026-11-06T18:31:14.317247Z     debug   envoy http external/envoy/source/common/http/conn_manager_impl.cc:2135  [Tags:
"ConnectionId":"1146","StreamId":"4067863431051959824"] Codec completed encoding stream.
```

The lines come from the logger `http` of Envoy, with the id of the connection and of the stream. Clean up and reset:

```
istioctl proxy-config log deploy/productpage-v1 -r
kubectl delete pod curl-test
```

Set the level for good

The values of the `istiod` Helm chart (`istiod-1.29.9`). In our lab they have the defaults:

```
helm -n istio-system get values istiod -a | grep -E "^\s+(logging:|level:|logLevel:|componentLogLevel:|logAsJson:)"
```

```
    logAsJson: false
    logging:
      level: default:info
      componentLogLevel: misc:error
      logLevel: warning
```

Helm value	Default	What it sets
`global.logging.level`	`default:info`	the log level of `istiod` **and** of the `pilot-agent` in every new sidecar
`global.proxy.logLevel`	`warning`	the level of Envoy in new sidecars
`global.proxy.componentLogLevel`	`misc:error`	levels of single Envoy components in new sidecars
`global.logAsJson`	`false`	log lines as JSON

We checked with `helm template` : `--set-string 'global.logging.level=default:info\,ads:debug'` gives the argument `--log_output_level=default:info,ads:debug` in the Deployment of `istiod` . The sidecar values are put in when the pod is created, so they only apply to **new** pods (see sidecar injection).

For one pod, the injection template of our lab knows the annotations `sidecar.istio.io/logLevel` , `sidecar.istio.io/componentLogLevel` and `sidecar.istio.io/agentLogLevel` . We did not try them on a pod.

Rules of thumb

Istio itself gives no advice on log levels (the page *Component Logging* lists the levels and how to set them). These rules are our experience:

- Keep the defaults in the Helm values, in Git.
- Raise a level only for the scope or logger you need, on the one pod or on `istiod` , and **always reset it** afterwards.
- Do not use `global.logging.level` for debugging: it also changes the `pilot-agent` of every new sidecar.
- If the logs go to a central system, think about `global.logAsJson` .

Next: ControlZ

ControlZ: the debug page of istiod

ControlZ is the introspection interface of `istiod` : a small web interface that shows the inner state of the running program. The Istio documentation calls it "a flexible introspection framework" that "makes it easy to inspect and manipulate the internal state of an istiod instance". It listens on port `9876` .

In our lab it listens on `127.0.0.1` only, so it cannot be reached from outside the pod. You reach it with a port-forward. You already use it: `istioctl admin log` talks to `istiod` through ControlZ (the command has the flag `--ctrlz_port` , default `9876`), see Logs and log levels.

Open it

```
istioctl dashboard controlz deployment/istiod.istio-system --browser=false
```

```
http://localhost:9876
skipping opening a browser
```

The command keeps the port-forward open until you press `Ctrl-C` . With a browser on your own computer, leave out `--browser=false` . On the bastion host there is no browser: run the port-forward yourself in one terminal and use `curl` in a second terminal.

```
kubectl -n istio-system port-forward deploy/istiod 9876:9876
```

The pages

Page	Shows
`/scopez/`	**Logging Scopes** of `istiod`: name, description and level, the scopes that `istioctl admin log` lists
`/envz/`	**Environment Variables** of the process
`/argz/`	**Command-Line Arguments**
`/memz/`	**Memory Usage**
`/procz/`	**Process Info**
`/versionz/`	**Version Info**
`/signalz/`	**Signals** (we did not look at its content)

`/scopez/` , `/envz/` , `/memz/` and `/procz/` also exist as JSON: `/scopej/` , `/envj/` , `/memj/` and `/procj/` .

Try it

With the port-forward running in the other terminal:

```
curl -s localhost:9876/procj/
curl -s localhost:9876/scopej/ | head -c 250
curl -s localhost:9876/envj/ | grep -o -E '\{"name":"PILOT_ENABLE_WORKLOAD_ENTRY_AUTOREGISTRATION","value":"[^"]*"\}'
curl -s localhost:9876/memj/ | head -c 120
```

```
{"egid":1337,"euid":1337,"gid":1337,"groups":[1337],"pid":1,"ppid":0,"uid":1337,"wd":"istiod-77ddcf67cd-
2djvq","hostname":"istiod-77ddcf67cd-2djvq","tempdir":"/tmp","threads":10,"goroutines":962}
[{"name":"ads","description":"ads debugging","output_level":"info","stack_trace_level":"none","log_callers":false},
{"name":"adsc","description":"adsc debugging","output_level":"info","stack_trace_level":"none","log_callers":false},
{"name":"analysis",
{"name":"PILOT_ENABLE_WORKLOAD_ENTRY_AUTOREGISTRATION","value":"true"}
{"Alloc":35280184,"TotalAlloc":712473592,"Sys":79653144,"Lookups":0,"Mallocs":8053762,"Frees":7810523,"HeapAlloc":352801
```

The third command answers a question that comes up often: **is a setting really switched on in `istiod` ?** The variable `PILOT_ENABLE_WORKLOAD_ENTRY_AUTOREGISTRATION` is set by the VM exercise. The same works for `ISTIO_DUAL_STACK` and any other variable of `istiod` .

What it is good for when you debug

- **Which settings does the running `istiod` really have?** `/envz/` and `/argz/` show the environment and the start arguments, not what you meant to configure.
- **Which log scopes are there, and which level do they have?** `/scopez/` , or `istioctl admin log` .
- **How much memory and how many goroutines does `istiod` use?** `/memz/` and `/procz/` , for example while you look at the resources of `istiod` .

The environment page shows **all** environment variables of `istiod` . Look at them before you share the output.

`istioctl dashboard` can open more than ControlZ (listed in our lab: `controlz` , `envoy` , `grafana` , `istiod-debug` , `jaeger` , `kiali` , `prometheus` , `proxy` , `skywalking` , `zipkin`). We only used `controlz` .

Previous: Logs and log levels